

Contents

WHAT IS LLM ?	3
Transformers	5
11. Stacked Transformer Layers	11
12. Output Linear Layer	11
13. Softmax	12
Transformers Main Parts	13
Tokenization	14
4 Stages of Training LLMs from Scratch	22
Stage 0: Randomly Initialized Model	23
Stage 1: Pre-training	24
Stage 2: Instruction Fine-Tuning	24
Stage 3: Preference Fine-Tuning (PFT)	25
Stage 4: Reasoning Fine-Tuning	26
Quick Recap	26
A Couple of Things Worth Adding	27
1. Conditional Probability — The Foundation	27
2. How This Applies to LLMs	27
3. What the Model Actually Learns	28
4. The Problem With Always Picking the Best Word	28
5. Temperature and Sampling	28
Interview-Ready Notes (Simple 2-Line Answers)	28
One-Line Summary	29
Overview	30
7 LLM Generation Parameters	31
0. The Big Picture (say this first if asked to explain generation)	35
1. Max Tokens	36
2. Temperature	36
3. Top-k	37
4. Top-p (Nucleus Sampling)	37
5. Frequency Penalty	38
6. Presence Penalty	38
7. Stop Sequences	39

Quick Comparison Table (great for a whiteboard answer).....	39
Likely Interview Questions + Simple Answers.....	40
4 LLM Text Generation Strategies	41
Closing summary (with analogy).....	44
3 Techniques to Train An LLM Using Another LLM	44
Quick way to wrap it up	48
Transformer vs. Mixture of Experts (MoE) in LLMs	49
Core Idea	50
How Experts Get Selected — The Router	50
Challenge 1: Expert Imbalance (Rich-Get-Richer Problem)	51
Challenge 2: Uneven Token Load.....	51
Key Outcome.....	51
Simple Explanation with Analogy (Interview-Ready).....	51
Analogy: A Hospital With Specialist Doctors.....	51
How I'd Explain It Out Loud in an Interview.....	52
Quick Closing Line for Interviews.....	52

WHAT IS LLM ?

What is an LLM?

Imagine someone begins a sentence:



"Once upon a..."

You naturally think "time."

Or they say:

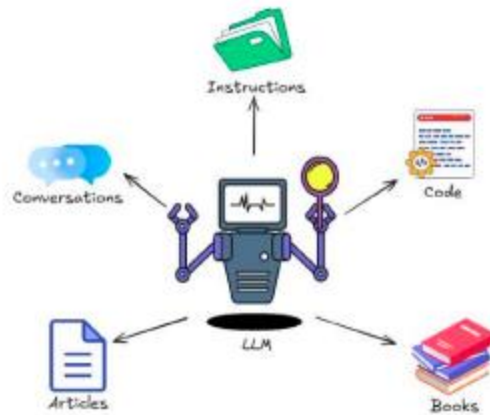
"The capital of France is..."

You immediately think "Paris."



This simple act of predicting what comes next is the foundation of how large language models (LLMs) operate.

They learn to make these predictions by reading enormous amounts of text: books, articles, scientific papers, code, conversations, and instructions.



With enough exposure, the model becomes remarkably good at continuing any piece of text in a coherent, meaningful way.

At the technical level, an LLM processes text in small units called tokens. A token may be a word, part of a word or even punctuation.



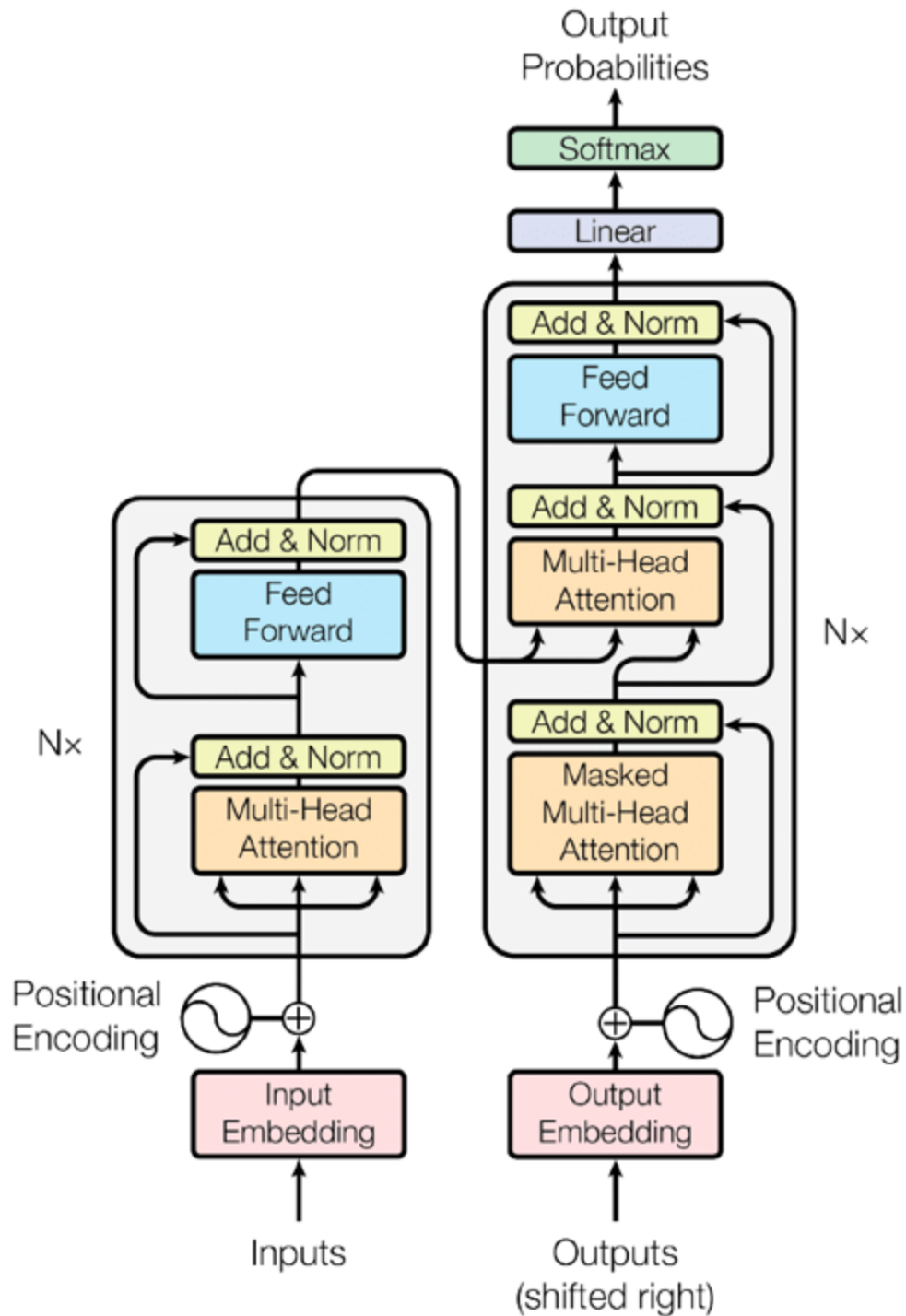
The model looks at the tokens so far and predicts the next one. Repeating this process generates full answers, explanations, or code.

Everything an LLM does from summarizing a document, generating a function or explaining a concept emerges from choosing the next token that best fits the patterns it has learned.

Transformers

To formalise, a large language model is a Transformer-based neural network trained on massive text corpora to predict the next token in a sequence and through this process acquires the ability to understand, generate and reason with human language.

For indepth notes refer finetuning notes word doc prepared by me



A **Transformer** is a deep learning architecture that processes sequences using **Attention Mechanism** instead of recurrence. It is the core architecture used in modern GenAI models.

Sentence:

“The cat sat on the mat”

We will see **what happens to this sentence inside the transformer**.

1. Tokenization

Purpose (one line):

Break the sentence into smaller pieces called **tokens**.

What happens

"The cat sat on the mat"

becomes

["The", "cat", "sat", "on", "the", "mat"]

Some models may convert them to subwords like

["The", "cat", "sat", "on", "the", "mat"]

Now each token will be processed individually.

2. Input Embedding

Purpose (one line):

Convert tokens into **numeric vectors** so the neural network can process them.

What happens

Each word becomes a vector.

Example (simplified):

The → [0.21, 0.54, 0.11, ...]

cat → [0.33, 0.19, 0.72, ...]

sat → [0.48, 0.67, 0.12, ...]

These vectors represent **semantic meaning** of words.

3. Positional Encoding

Purpose (one line):

Add information about **word order** because transformers process tokens in parallel.

What happens

The model adds position information:

The → embedding + position(1)

cat → embedding + position(2)

sat → embedding + position(3)

Now the model knows:

cat comes after The

sat comes after cat

Without this, the model would treat the sentence as a **bag of words**.

4. Linear Projection → Query, Key, Value

Purpose (one line):

Create **Query, Key, Value vectors** used to calculate attention.

Each token vector is converted into:

Query (Q)

Key (K)

Value (V)

Example:

cat → Q_cat, K_cat, V_cat

sat → Q_sat, K_sat, V_sat

These vectors help the model decide **which words should pay attention to which words**.

5. Scaled Dot Product Attention

Purpose (one line):

Compute **how strongly each word relates to every other word**.

Example:

When processing **sat**, the model checks which words are important.

Attention scores may look like:

Word Attention from "sat"

The	0.1
cat	0.6
sat	0.2
on	0.05
mat	0.05

Meaning:

"sat" strongly relates to "cat"

So the model understands:

cat is the one sitting

This is the core idea of **Self-Attention**.

6. Multi-Head Attention

Purpose (one line):

Allow the model to capture **multiple relationships simultaneously**.

Instead of one attention calculation, the model runs **many attention heads**.

Example heads might learn:

- Head 1 → subject relationship
- Head 2 → verb relationship
- Head 3 → positional relation
- Head 4 → semantic similarity

Example:

Head1: cat $\leftrightarrow$ sat
Head2: sat $\leftrightarrow$ mat
Head3: cat $\leftrightarrow$ the

All heads are combined to create a richer representation.

7. Add & Residual Connection

Purpose (one line):

Add the original input back to prevent losing information.

Formula idea:

$\text{output} = \text{attention_output} + \text{original_input}$

This helps deep networks train better.

8. Layer Normalization

Purpose (one line):

Normalize values to stabilize training and speed convergence.

It keeps values within a controlled range so the network doesn't explode or vanish.

9. Feed Forward Neural Network (FFN)

Purpose (one line):

Apply nonlinear transformations to refine token representations.

Each token passes through a small neural network:

$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$

Example improvement:

"sat" representation now strongly connects with "cat"

The model better understands **sentence meaning**.

10. Add & LayerNorm Again

Purpose (one line):

Stabilize and preserve information after the feed-forward step.

Again:

$\text{output} = \text{FFN_output} + \text{previous_input}$

Then normalization is applied.

11. Stacked Transformer Layers

Purpose (one line):

Repeat the attention + FFN process multiple times to build deeper understanding.

Typical models stack:

6 layers
12 layers
24 layers
96 layers (in large LLMs)

Each layer captures **more complex relationships**.

Example progression:

Layer 1 → basic word relationships
Layer 4 → subject-verb structure
Layer 8 → sentence meaning
Layer 12 → deeper context

12. Output Linear Layer

Purpose (one line):

Convert the final hidden representation into **scores for each vocabulary word**.

Example predicted scores for next word:

mat → 0.52
floor → 0.22

chair → 0.10
table → 0.05

13. Softmax

Purpose (one line):

Convert scores into probabilities.

Example:

mat = 52%
floor = 22%
chair = 10%

The model predicts:

"mat"

Full Transformer Flow (Simple)

Sentence
↓
Tokenization
↓
Input Embedding
↓
Positional Encoding
↓
Q, K, V creation
↓
Self Attention
↓
Multi Head Attention
↓
Add & Norm
↓
Feed Forward Network
↓
Add & Norm
↓
Repeat layers
↓
Linear Layer
↓
Softmax
↓
Next token prediction

Component	Purpose
Tokenization	Split text into tokens
Embedding	Convert tokens to vectors
Positional Encoding	Preserve word order
Self Attention	Learn relationships between words
Query/Key/Value	Calculate attention relevance
Multi Head Attention	Learn multiple relationships simultaneously
Add & Norm	Stabilize training
Feed Forward	Extract deeper features
Transformer Layers	Build hierarchical understanding
Softmax Output	Generate prediction

One Sentence Summary

A **Transformer** processes text by converting tokens into vectors and repeatedly applying **Self-Attention** and feed-forward layers so each word understands its relationship with every other word.

Transformers Main Parts

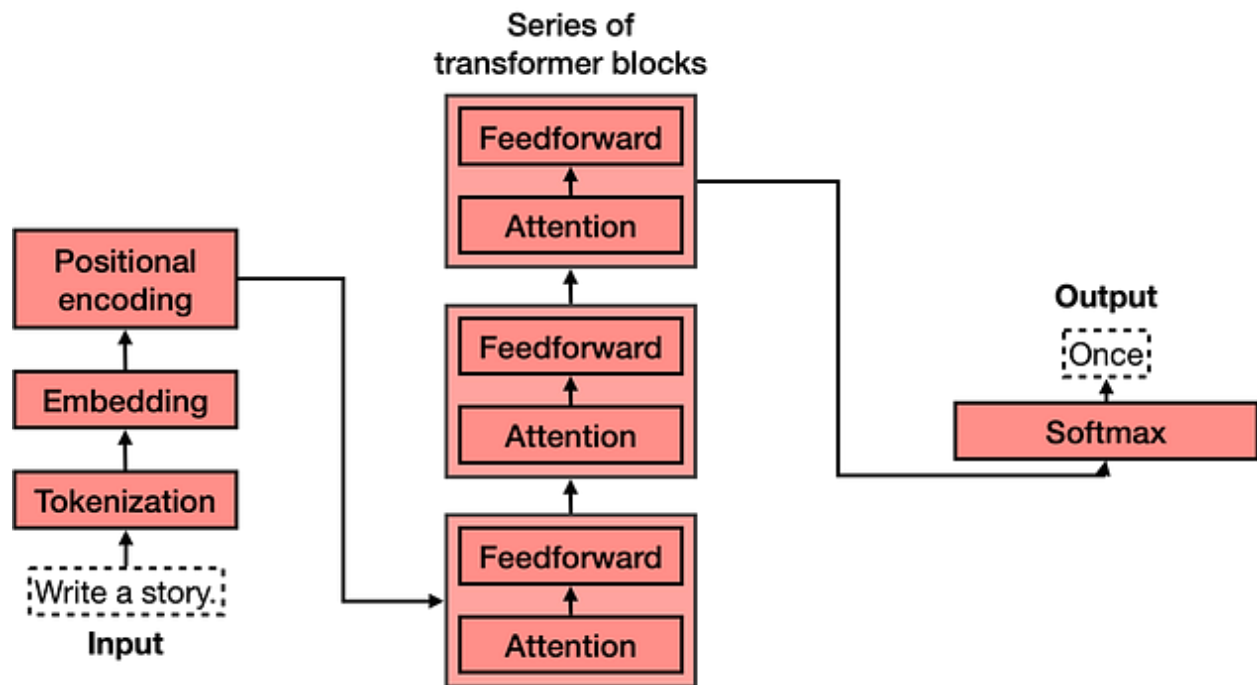
<https://medium.com/@amanatulla1606/transformer-architecture-explained-2c49e2257b4c>

Now that we know what transformers do, let's get to their architecture. If you've seen the architecture of a transformer model, you may have jumped in awe like I did the first time I saw it, it looks quite complicated! However, when you break it down into its most important parts, it's not so bad. The transformer has 4 main parts:

- Tokenization

- Embedding
- Positional encoding
- Transformer block (several of these)
- Softmax

The fourth one, the transformer block, is the most complex of all. Many of these can be concatenated, and each one contains two main parts: The attention and the feedforward components.



Tokenization

Tokenization is the most basic step. It consists of a large dataset of tokens, including all the words, punctuation signs, etc. The tokenization step takes every word, prefix, suffix, and punctuation signs, and sends them to a known token from the library.

Tokenization



Embedding

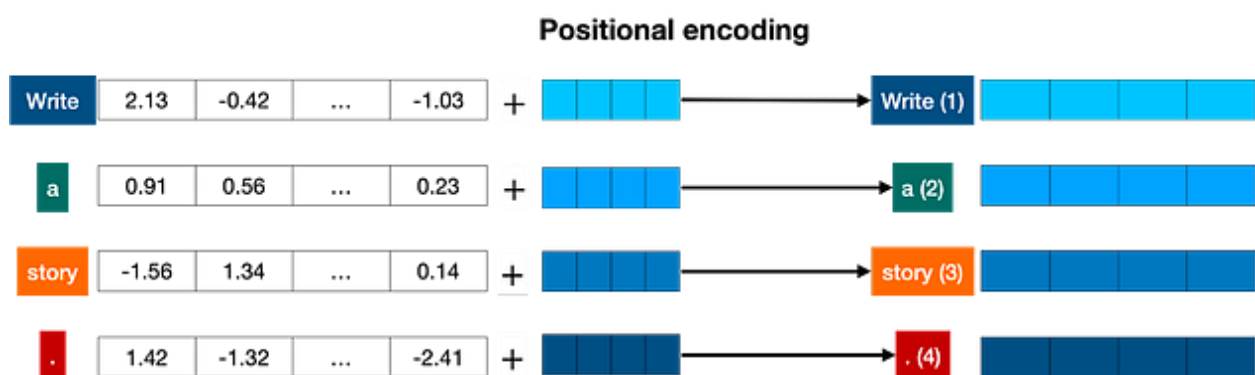
Once the input has been tokenized, it's time to turn words into numbers. For this, we use an embedding. In a previous chapter you learned about how text embeddings send every piece of text to a vector (a list) of numbers. If two pieces of text are similar, then the numbers in their corresponding vectors are similar to each other (componentwise, meaning each pair of numbers in the same position are similar). Otherwise, if two pieces of text are different, then the numbers in their corresponding vectors are different.

Embedding



Positional encoding

Once we have the vectors corresponding to each of the tokens in the sentence, the next step is to turn all these into one vector to process. The most common way to turn a bunch of vectors into one vector is to add them, componentwise. That means, we add each coordinate separately. For example, if the vectors (of length 2) are $[1,2]$, and $[3,4]$, their corresponding sum is $[1+3, 2+4]$, which equals $[4, 6]$. This can work, but there's a small caveat. Addition is commutative, meaning that if you add the same numbers in a different order, you get the same result. In that case, the sentence "I'm not sad, I'm happy" and the sentence "I'm not happy, I'm sad", will result in the same vector, given that they have the same words, except in different order. This is not good. Therefore, we must come up with some method that will give us a different vector for the two sentences. Several methods work, and we'll go with one of them: positional encoding. Positional encoding consists of adding a sequence of predefined vectors to the embedding vectors of the words. This ensures we get a unique vector for every sentence, and sentences with the same words in different order will be assigned different vectors. In the example below, the vectors corresponding to the words "Write", "a", "story", and "." become the modified vectors that carry information about their position, labeled "Write (1)", "a (2)", "story (3)", and ". (4)".



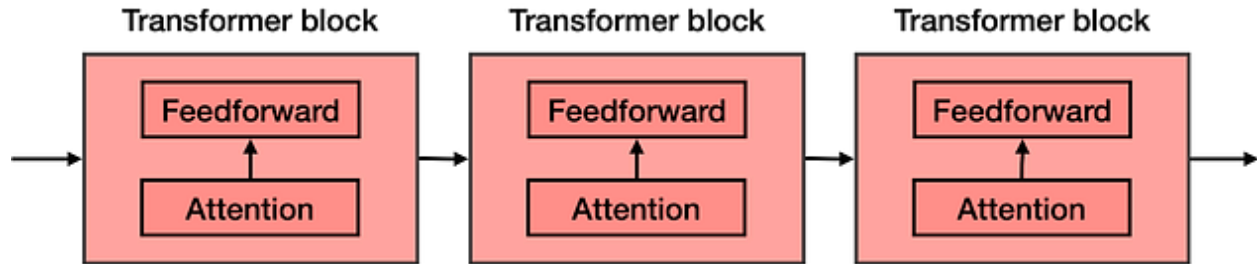
Transformer block

Let's recap what we have so far. The words come in and get turned into tokens (tokenization), tokenized words are turned into numbers (embeddings), then order gets taken into account (positional encoding). This gives us a vector for every token that we input to the model. Now, the next step is to predict the next word in this sentence. This is done with a really really large neural network, which is trained precisely with that goal, to predict the next word in a sentence.

We can train such a large network, but we can vastly improve it by adding a key step: the attention component. Introduced in the seminal paper *Attention is All you Need*, it is one of the key ingredients in transformer models, and one of the reasons they work so well. Attention is explained in the previous section, but for now, imagine it as a way to add context to each word in the text.

The attention component is added at every block of the feedforward network. Therefore, if you imagine a large feedforward neural network whose goal is to predict the next word, formed by several blocks of smaller neural networks, an attention component is added to each one of these blocks. Each component of the transformer, called a transformer block, is then formed by two main components:

- The attention component.
- The feedforward component.



Attention

The next step is attention. As you learned attention mechanism deals with a very important problem: the problem of context. Sometimes, as you know, the same word can be used with different meanings. This tends to confuse language models, since an embedding simply sends words to vectors, without knowing which definition of the word they're using.

Attention is a very useful technique that helps language models understand the context. In order to understand how attention works, consider the following two sentences:

- Sentence 1: The bank of the river.
- Sentence 2: Money in the bank.

As you can see, the word 'bank' appears in both, but with different definitions. In sentence 1, we are referring to the land at the side of the river, and in the second one to the institution that holds money. The computer has no idea of this, so we need to somehow inject that knowledge into it. What can help us? Well, it seems that the other words in the sentence can come to our rescue. For the first sentence, the words 'the', and 'of' do us no good.

But the word 'river' is the one that is letting us know that we're talking about the land at the side of the river. Similarly, in sentence 2, the word 'money' is the one that is helping us understand that the word 'bank' is now referring to the institution that holds money.



In short, what attention does is it moves the words in a sentence (or piece of text) closer in the word embedding. In that way, the word "bank" in the sentence "Money in the bank" will be moved closer to the word "money".

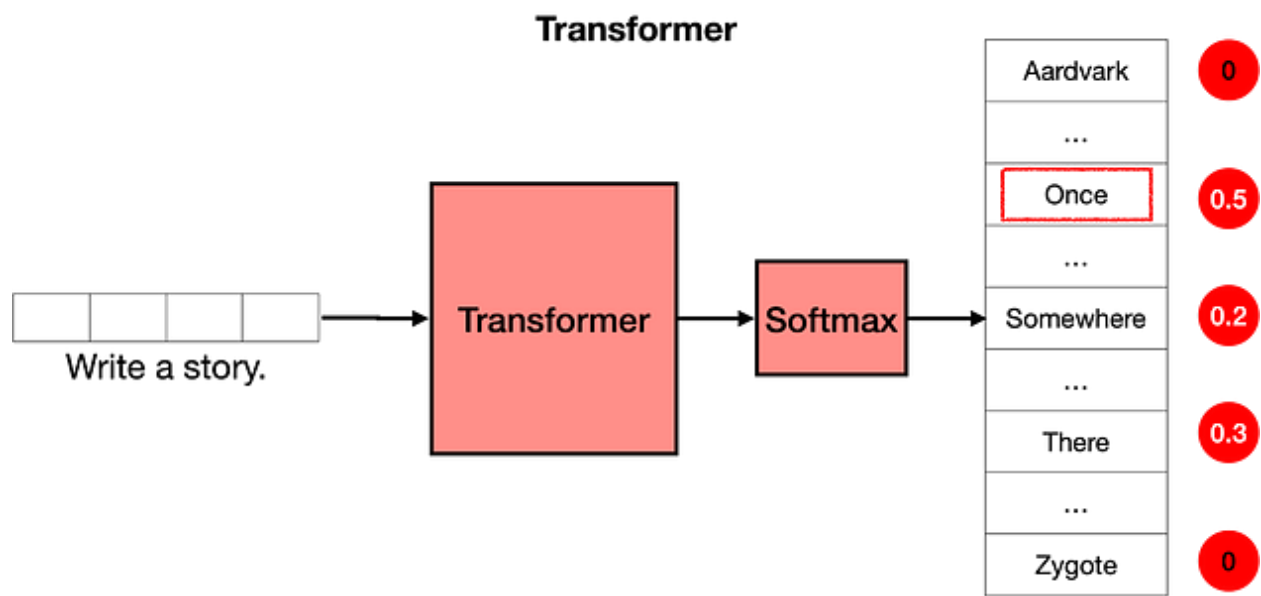
Equivalently, in the sentence "The bank of the river", the word "bank" will be moved closer to the word "river". That way, the modified word "bank" in each of the two sentences will carry some of the information of the neighboring words, adding context to it.

The attention step used in transformer models is actually much more powerful, and it's called **multi-head attention**. In multi-head attention, several different embeddings are used to modify the vectors and add context to them. Multi-head attention has helped language models reach much higher levels of efficacy when processing and generating text.

The Softmax Layer

Now that you know that a transformer is formed by many layers of transformer blocks, each containing an attention and a feedforward layer, you can think of it as a large neural network that predicts the next word in a sentence. The transformer outputs scores for all the words, where the highest scores are given to the words that are most likely to be next in the sentence.

The last step of a transformer is a softmax layer, which turns these scores into probabilities (that add to 1), where the highest scores correspond to the highest probabilities. Then, we can sample out of these probabilities for the next word. In the example below, the transformer gives the highest probability of 0.5 to “Once”, and probabilities of 0.3 and 0.2 to “Somewhere” and “There”. Once we sample, the word “once” is selected, and that’s the output of the transformer.



The softmax layer turns the scores into probabilities, and these are used to pick the next word in the text.

Now what? Well, we repeat the step. We now input the text “Write a story. Once” into the model, and most likely, the output will be “upon”. Repeating this step again and again, the transformer will end up writing a story, such as “Once upon a time, there was a ...”.

Post Training

Now that you know how transformers work, we still have a bit of work to do. Imagine the following: You ask the transformer “What is the capital of Algeria?”. We would love for it to answer “Algiers”, and move on. However, the transformer is trained on the entire internet. The internet is a big place, and it’s not necessarily the best question/answer repository. Many pages, for example, would have long lists of questions without answers. In this case, the next sentence after “What is the capital of Algeria?” could be another question, such as “What is the population of Algeria?”, or “What is the capital of Burkina Faso?”. The transformer is not a human who thinks about their responses, it simply mimics what it sees on the internet (or any dataset that has been provided). So how do we get the transformer to answer questions?

The answer is post-training. In the same way that you would teach a person to do certain tasks, you can get a transformer to perform tasks. Once a transformer is trained on the entire internet, then it is trained again on a large dataset which corresponds to lots of questions and their respective answers. Transformers (like humans), have a bias towards the last things

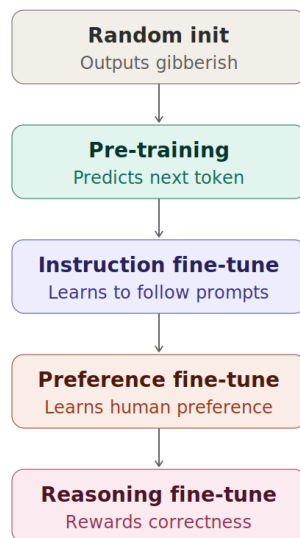
they've learned, so post-training has proven a very useful step to help transformers succeed at the tasks they are asked to.

Post-training also helps with many other tasks. For example, one can post-train a transformer with large datasets of conversations, in order to help it perform well as a chatbot, or to help us write stories, poems, or even code.

4 Stages of Training LLMs from Scratch

A simplified explanation, based on "4 Stages of Training LLMs from Scratch" by Avi Chawla (Daily Dose of Data Science)

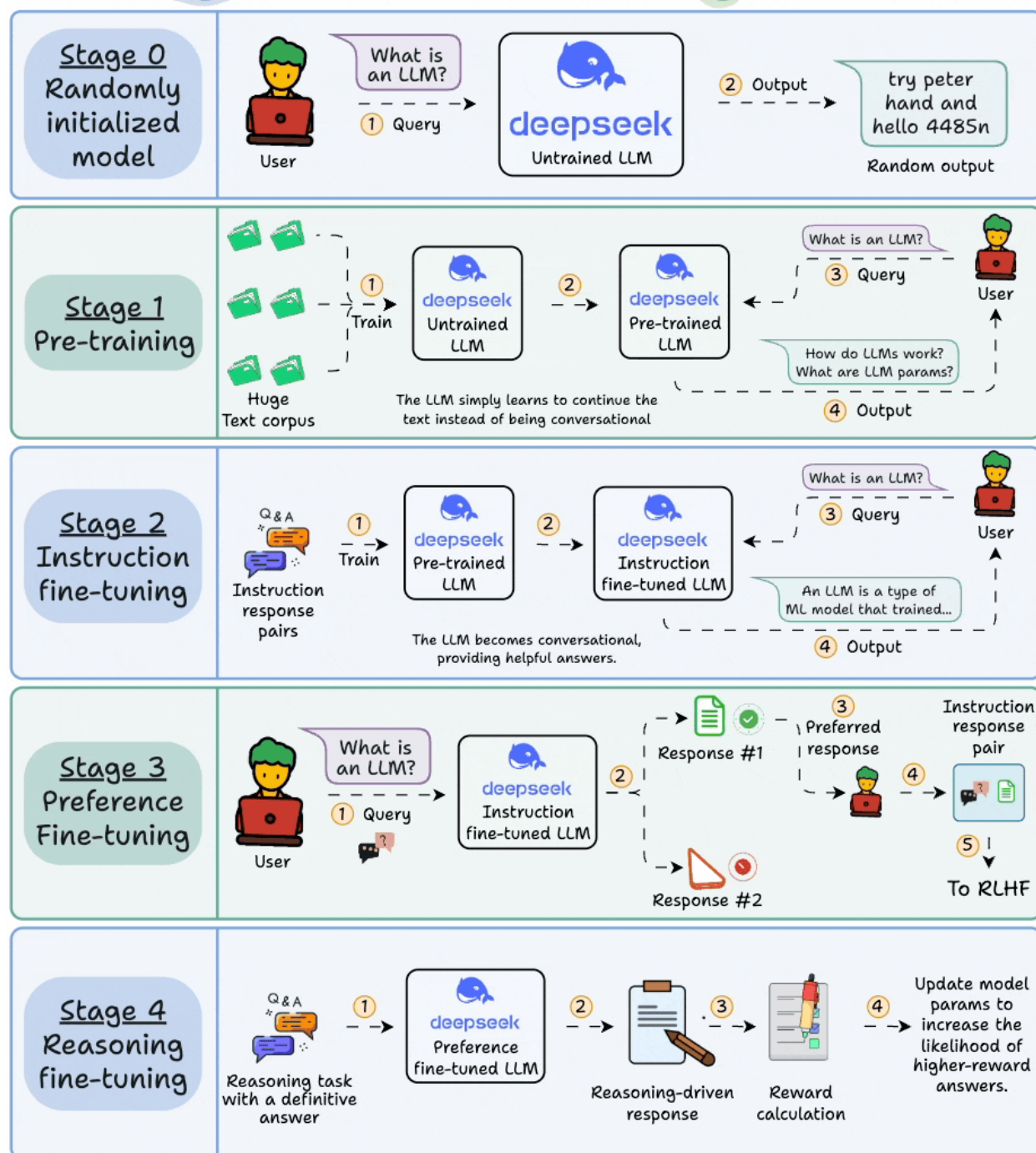
<https://blog.dailydoseofds.com/p/4-stages-of-training-llms-from-scratch>



Each stage builds on the model produced by the one before it

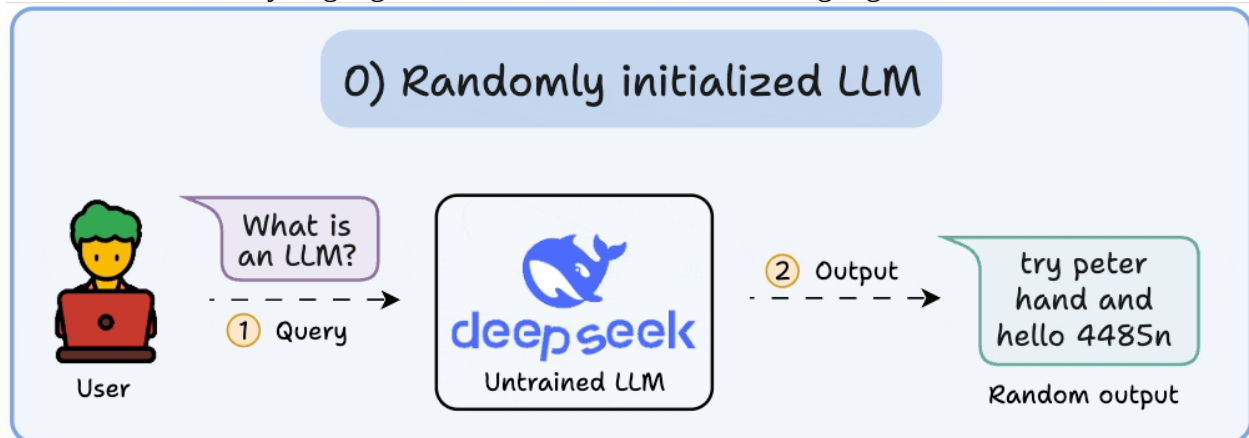
Building a large language model isn't just one training run. It happens in stages, and each stage builds on the one before it. Here are the 4 main stages, in simple words

4 stages of LLM Training



Stage 0: Randomly Initialized Model

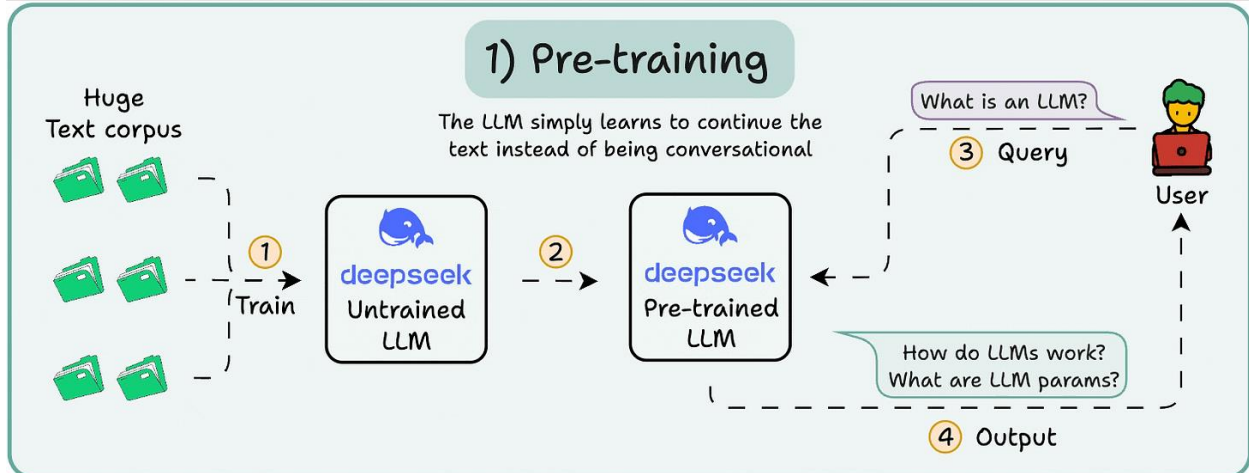
Before any training, the model knows nothing. Its weights are just random numbers. If you ask it "What is an LLM?" you get gibberish, like random words strung together.



Stage 1: Pre-training

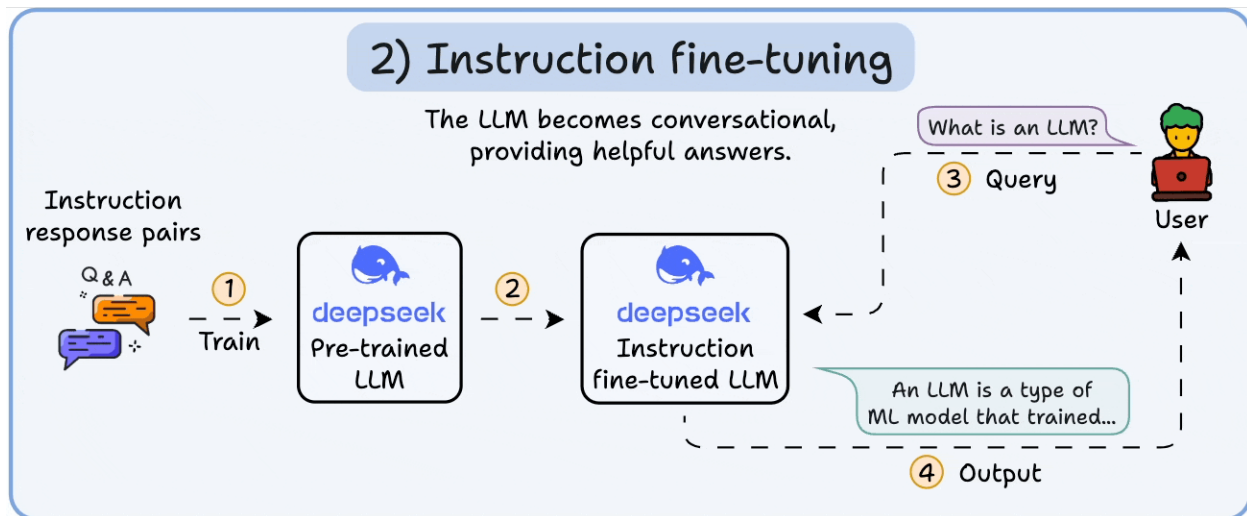
The model is trained on huge amounts of text from the internet. Its only job is to guess the next word, over and over. Doing this billions of times teaches it grammar, facts, and how language works.

Problem: it's not good at conversation yet. Ask it a question, and it may just continue the sentence instead of answering — because that's all it learned to do: continue text.



Stage 2: Instruction Fine-Tuning

Now the model is trained again, this time on examples of instructions paired with good responses (question and answer pairs). This teaches it to actually follow what you ask and reply properly, instead of just continuing text.



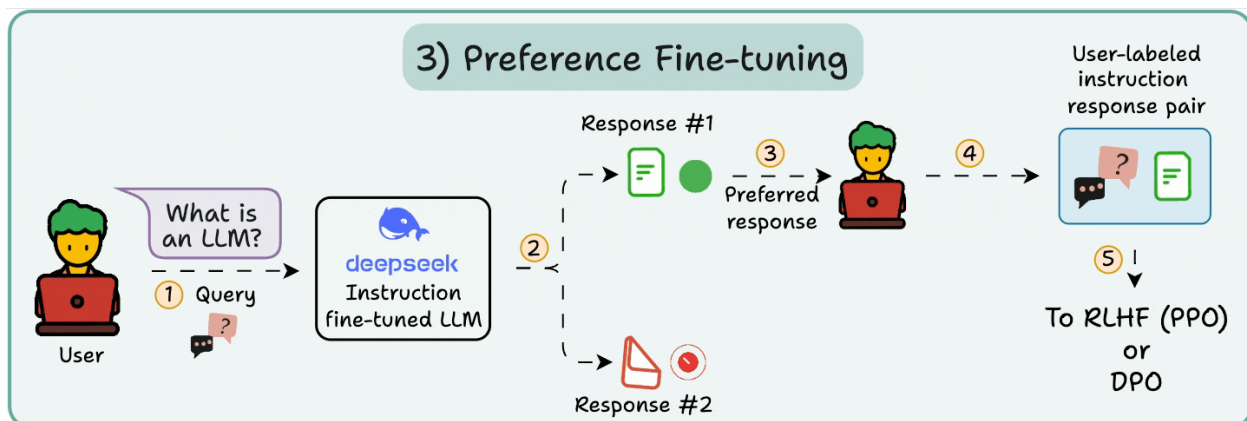
After this stage, the model can:

- Answer questions
- Summarize text
- Write code

At this point, the model has learned from pretty much all the internet text and human-written instruction examples available. To improve further, the next stages use reinforcement learning (RL) instead of just more examples.

Stage 3: Preference Fine-Tuning (PFT)

You may have seen an AI chat app show two answers and ask "which one do you prefer?" That click isn't just feedback — it's valuable training data about human preference.



Here's how it works:

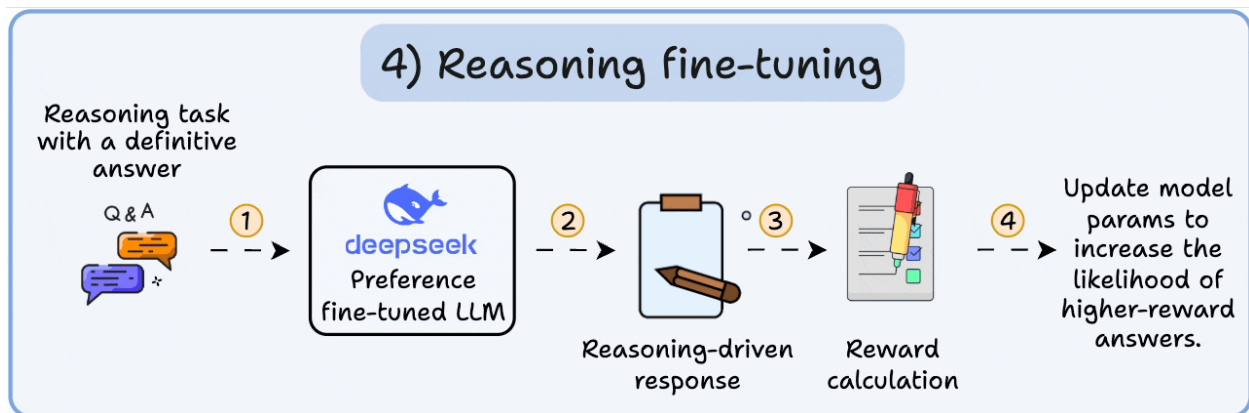
- The model gives two different answers to the same question.
- A person picks the better one.
- A separate "reward model" learns to predict which answers humans prefer.

- The main model is then updated using reinforcement learning to produce more answers like the ones humans preferred.

This whole process is called RLHF — Reinforcement Learning with Human Feedback. It helps the model give better answers even when there's no single "correct" answer, like tone, helpfulness, or safety. (The classic algorithm used here is called PPO. Many newer models now use a simpler alternative called DPO.)

Stage 4: Reasoning Fine-Tuning

For tasks like math or logic, there's usually one clear correct answer and clear steps to get there. So instead of needing human preference, we can just check: was the answer right or wrong?



Steps:

- The model generates an answer.
- The answer is compared with the known correct answer.
- If it's correct, the model gets a reward, and it's trained to do more of that.

This approach is called RLVR — Reinforcement Learning with Verifiable Rewards, since the reward is based on checking correctness rather than human opinion. A well-known technique here is GRPO, used by DeepSeek.

Those were the 4 stages of training an LLM.

- Start with a randomly initialized model.
- Pre-train it on large-scale corpora.
- Use instruction fine-tuning to make it follow commands.
- Use preference & reasoning fine-tuning to sharpen responses.

Quick Recap

- Start with a randomly initialized model that knows nothing.
- Pre-train it on huge text data so it learns language and facts.
- Instruction fine-tune it so it follows commands and answers properly.
- Preference fine-tune it (RLHF) so its answers match what humans actually prefer.
- Reasoning fine-tune it (RLVR) so it gets better at tasks with a clear right answer, like math.

A Couple of Things Worth Adding

The original blog covers the core 4 stages accurately — this is the standard framework used across the industry. A couple of small additions for a fuller picture:

- PPO isn't the only algorithm for preference fine-tuning anymore — DPO (Direct Preference Optimization) is a simpler, increasingly common alternative.
- In practice, companies often loop back between stages (e.g. more preference tuning after reasoning tuning) rather than doing each stage just once.

How Do LLMs Work?

A simplified explanation, based on "How Do LLMs Work?" by Avi Chawla (Daily Dose of Data Science)

1. Conditional Probability — The Foundation

Conditional probability just means: the chance of one thing happening, given that something else already happened.

Example: If it's cloudy (event B), it's more likely to rain (event A). We write this as $P(A|B)$ — "probability of A given B." Clouds don't guarantee rain, but they raise the chance.

2. How This Applies to LLMs

An LLM's core job is to predict the next word in a sentence. This is exactly a conditional probability problem: given the words so far (the context), what's the most likely next word?

For every possible next word, the model calculates a probability — how likely that word is to come next, given the context. The word (or words) with high probability are strong candidates for what comes next.

3. What the Model Actually Learns

During training, the model learns a giant probability distribution over sequences of words — basically, patterns of which words tend to follow which other words, across huge amounts of text.

The model's trained weights are literally the parameters of this probability distribution. Training this (called pre-training) is done in a supervised way — the model repeatedly guesses the next word and gets corrected based on the real next word in the training text.

4. The Problem With Always Picking the Best Word

If the model always picks the single most probable next word every time (called greedy decoding), the output becomes repetitive and boring — and it kills the model's creativity.

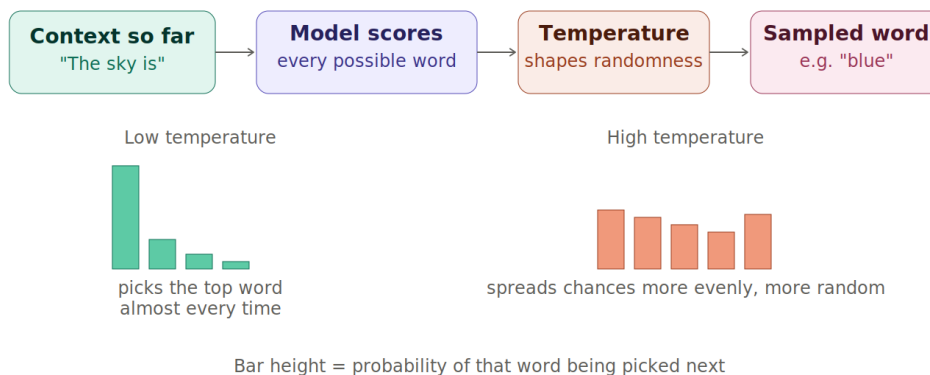
This is the problem that temperature and sampling are designed to fix.

5. Temperature and Sampling

Instead of always choosing the top word, the model samples — it randomly picks a word based on the probabilities, so less likely words still get a chance sometimes.

Temperature is a setting that controls how random this sampling is:

- Low temperature → probabilities stay concentrated on the top word(s). Output is close to always picking the best word (safe, focused, less creative).
- High temperature → probabilities spread out more evenly across many words. Output becomes more random and creative, but can be less coherent.



Interview-Ready Notes (Simple 2-Line Answers)

Conditional Probability:

The chance of one event happening given that another related event already happened. Written as $P(A|B)$ — probability of A given B.

Next-Word Prediction in LLMs:

The model treats next-word prediction as a conditional probability problem — given the context so far, it scores how likely each possible next word is.

What the Model Learns:

The model learns a huge probability distribution over word sequences during training. Its trained weights are the parameters of this distribution.

Greedy Decoding Problem:

Always picking the highest-probability word leads to repetitive, robotic output with no creativity.

Temperature:

A setting that controls how random the next-word sampling is — low temperature is focused and predictable, high temperature is more random and creative.

One-Line Summary

An LLM predicts the next word using conditional probability, and temperature controls how randomly it samples from those probabilities — low temperature plays it safe, high temperature gets more creative.

7 LLM Generation Parameters

Every response an LLM produces is shaped by a handful of parameters set at generation time. This guide walks through the 7 parameters that matter most, with an original diagram illustrating each one.

Overview

Knowing how to tune generation parameters is what separates a sharp, controlled LLM output from a haphazard one. Below is a quick-reference table, followed by a detailed, visual explanation of each parameter.

Parameter	Controls	Typical Use
Max tokens	Response length cap	Prevent runaway or cut-off generations
Temperature	Overall randomness	Low for factual tasks, high for creative tasks
Top-k	Candidate pool size (fixed count)	Limit sampling to a fixed number of top tokens
Top-p	Candidate pool size (probability mass)	Adaptive alternative to top-k
Frequency penalty	Repetition of already-frequent tokens	Reduce redundancy in summaries/reports
Presence penalty	Introduction of brand-new tokens	Encourage novelty in brainstorming
Stop sequences	Hard stop trigger	Enforce clean structured output boundaries

7 LLM Generation Parameters

Every generation from an LLM is shaped by parameters under the hood.

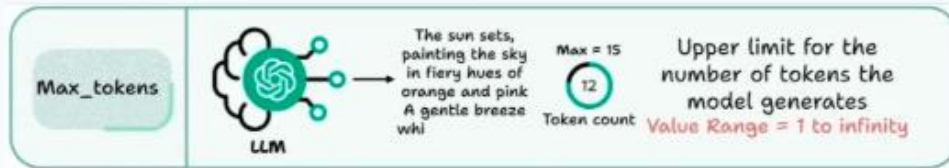
Knowing how to tune is important so that you can produce sharp and more controlled outputs.

Here are the 7 levers that matter most:

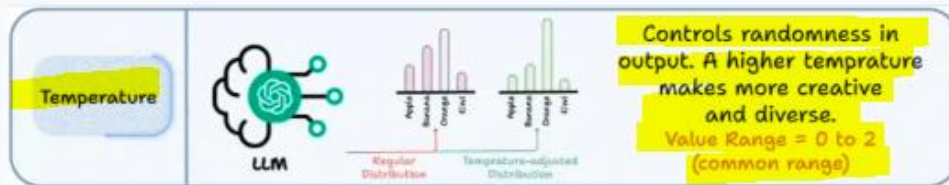
7 LLM Generation parameters

Max_tokens	 The sun sets, painting the sky in fiery hues of orange and pink. A gentle breeze whispers through the trees. Max = 15 Token count 15	Upper limit for the number of tokens the model generates Value Range = 1 to infinity
Temperature	 Regular Distribution Temperature-adjusted Distribution	Controls randomness in output. A higher temperature makes more creative and diverse. Value Range = 0 to 2 (common range)
Top_p	 top_p = 10%	Controls probability distribution is considered when sampling tokens Value Range = 0 to 1
Top_k	 top_k = 2	Limits the number of top probable tokens to sample from Value Range = 1 to infinity
Frequency penalty	 Dogs love to play and run and chase and bark and nap, and	Penalizes token repetition based on frequency. Positive values reduce repetition Value Range = -2 to 2
Presence penalty	 Puppies nap. Kittens play. The sun shines. A bird sings. The day is perfect.	Encourages the model to use new tokens that haven't been generated Value Range = -2 to 2
Stop	 There are some words restricted or forbidden, such as gambling.	A list of tokens where the model will stop generating further tokens Value Range = Custom list

- **Max tokens**

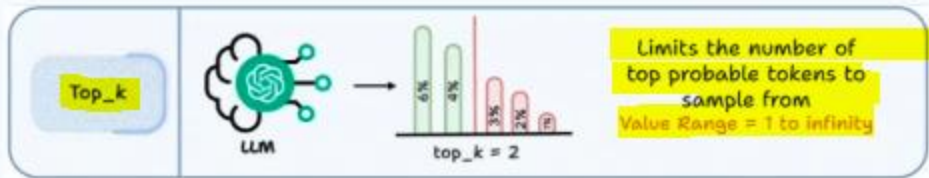


- This is a hard cap on how many tokens the model can generate in one response.
- Too low → truncated outputs; too high → could lead to wasted compute.
- **Temperature (covered in detail here):**



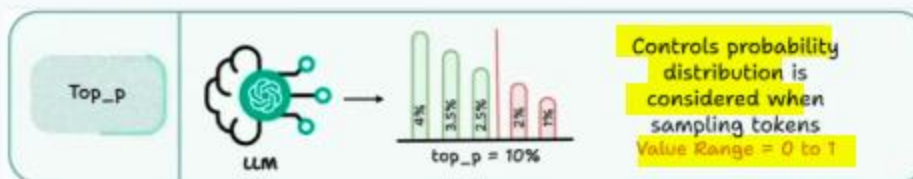
- Governs randomness. Low temperature (~0) makes the model deterministic.
- Higher temperature (0.7–1.0) boosts creativity, diversity, but also noise.
- Use case: lower for QA/chatbots, higher for brainstorming/creative tasks.

- **Top-k:**



- The default way to generate the next token is to sample from all tokens, proportional to their probability.
- This parameter restricts sampling to the top k most probable tokens.
- Example: $k=5 \rightarrow$ model only considers 5 most likely next tokens during sampling.
- Helps enforce focus, but overly small k may give repetitive outputs.

- **Top-p (nucleus sampling):**



- Instead of picking from all tokens or top k tokens, model samples from a probability mass up to p .
- Example: $\text{top}_p=0.9 \rightarrow$ only the smallest set of tokens covering 90% probability are considered.
- More adaptive than top_k , useful when balancing coherence with diversity.

- **Frequency penalty:**



- Reduces likelihood of reusing tokens that have already appeared frequently.
- Positive values discourage repetition, negative values exaggerate it.
- Useful for summarization (avoid redundancy) or poetry (intentional repetition).

- **Presence penalty**



- Encourages the model to bring in new tokens not yet seen in the text.
- Higher values push for novelty, lower values make the model stick to known patterns.
- Handy for exploratory generation where diversity of ideas is valued.

LLM Generation Parameters — Simple Interview Notes

Plain-English explanations for each parameter. No formulas — just what it does, why it matters, and a one-line way to say it in an interview.

0. The Big Picture (say this first if asked to explain generation)

"When an LLM generates text, at every single step it's not just picking the 'best' next word — it's looking at a ranked list of possible next words, each with a confidence score. These parameters control how that list gets narrowed down and which word actually gets picked from it. A few of them also control when the model should stop talking."

That's really it. Everything below is a variation on one of those two ideas: *narrowing the choices* or *deciding when to stop*.

1. Max Tokens

One-liner: "The word limit for the response."

Simple explanation: It's a hard cutoff on how long the response can be. Once the model hits that limit, it stops immediately — even if it was in the middle of a sentence.

Why it matters:

- Too low → answer gets cut off awkwardly.
- Too high → you might pay for/wait for more than you need.

Interview soundbite:

"Max tokens is just a length cap — it doesn't change what the model says, only how much of it you let it finish saying."

2. Temperature

One-liner: "How random or safe the model's word choices are."

Simple explanation: Think of the model as having a shortlist of possible next words, each more or less "confident." Temperature controls how much it's willing to gamble on a less-obvious choice.

- **Low temperature** → always picks the safest, most obvious word. Predictable, focused, sometimes boring.
- **High temperature** → more willing to pick surprising, creative words. More interesting, but riskier — can go off the rails.

Analogy:

"It's like asking someone a question — at low temperature they give you the textbook answer every time. At high temperature they start improvising and going off-script."

When to use what:

- Low temperature → coding, facts, customer support.
- High temperature → brainstorming, creative writing, poetry.

Interview soundbite:

"Temperature controls the model's willingness to take risks with word choice — low is predictable, high is creative but noisier."

3. Top-k

One-liner: "Only consider the top K most likely next words."

Simple explanation: Before picking a word, the model narrows its options down to a fixed number — say, the top 10 most likely candidates — and only chooses from that shortlist. Everything else is thrown away, no matter how the rest of the list looks.

Analogy:

"It's like a hiring manager who always looks at exactly the top 5 resumes, no matter how many great candidates there really are or aren't."

Trade-off:

- Too small K → responses feel repetitive and robotic.
- Reasonable K → keeps things focused without being too rigid.

Interview soundbite:

"Top-k limits the model to picking from a fixed number of top candidates — it's a simple, blunt way to cut out unlikely words."

4. Top-p (Nucleus Sampling)

One-liner: "Only consider enough top words to cover, say, 90% of the confidence."

Simple explanation: Instead of a fixed number of words like top-k, top-p keeps adding the next most-likely words until their combined confidence adds up to a target percentage (e.g., 90%). Then it picks from just that group.

Why it's smarter than top-k: It automatically adjusts. If the model is very sure about the next word, the shortlist might just be 1–2 words. If the model is unsure, the shortlist naturally grows to include more options. Top-k can't do that — it always uses the same fixed count.

Analogy:

"Instead of always looking at exactly 5 resumes, the hiring manager keeps reading resumes until they've covered '90% of the good candidates' — sometimes that's 2 resumes, sometimes it's 15, depending on how competitive the pool is."

Interview soundbite:

"Top-p is the adaptive version of top-k — instead of a fixed count of words, it uses a fixed amount of confidence."

5. Frequency Penalty

One-liner: "Discourages the model from repeating the same words too much."

Simple explanation: The more a word has already been used in the response, the more the model gets discouraged from using it again. The penalty builds up the more times a word repeats.

Analogy:

"Like a teacher telling a student, 'You've already used the word "amazing" four times — try something else.' The more you overuse a word, the stronger the nudge to stop."

When to use it:

- Turn it up for summaries/reports where you don't want redundant phrasing.
- Turn it down (or make it negative) if some repetition is intentional, like in poetry.

Interview soundbite:

"Frequency penalty punishes words based on how *often* they've already shown up — the more repeats, the bigger the penalty."

6. Presence Penalty

One-liner: "Encourages the model to bring in new words/topics it hasn't used yet."

Simple explanation: This one doesn't care *how many times* a word has been repeated — it only checks *has this word shown up at all yet?* If yes, it gets a small, flat penalty, encouraging the model to explore something new instead.

How it's different from frequency penalty:

- Frequency penalty = "you've used this word 5 times, stop."
- Presence penalty = "you've used this word even once — try mixing in something new."

Analogy:

"It's like a brainstorming coach saying 'you already mentioned that idea — what's something you haven't brought up yet?' — it doesn't matter if you mentioned it once or five times, the nudge is the same."

When to use it: Great for brainstorming or idea generation, where you want breadth of topics rather than depth on one thing.

Interview soundbite:

"Presence penalty nudges the model toward new topics it hasn't touched yet, regardless of how many times it repeated something else."

7. Stop Sequences

One-liner: "Specific words/symbols that tell the model 'stop right here.'"

Simple explanation: You give the model a list of trigger strings (like `"\n\n"`, `"END"`, or `"}"`). The moment the model produces one of those, generation halts immediately — no matter what else it might have been about to say.

Why it matters: Super useful when you need clean, structured output — like JSON — where you don't want the model to add extra commentary after the actual answer is done.

Analogy:

"It's like giving someone a magic word — the second they say it, they have to stop talking, mid-thought if necessary."

Interview soundbite:

"Stop sequences are a hard, rule-based stop trigger — unlike max tokens, which is just a length cap, this stops generation based on *content*, not count."

Quick Comparison Table (great for a whiteboard answer)

Parameter	What it controls	Simple mental model
Max tokens	Response length	"Word limit"
Temperature	Randomness/creativity	"How much it gambles on unusual words"
Top-k	Candidate pool (fixed size)	"Only look at the top K options"

Parameter	What it controls	Simple mental model
Top-p	Candidate pool (adaptive size)	"Only look at options covering X% confidence"
Frequency penalty	Repetition (scales with count)	"Stop repeating that word so much"
Presence penalty	Topic diversity (flat, one-time)	"Bring in something new"
Stop sequences	Hard stop trigger	"Stop the moment you see this"

Likely Interview Questions + Simple Answers

Q: What's the difference between top-k and top-p?

"Top-k always looks at a fixed number of word choices. Top-p instead looks at however many words it takes to cover a target confidence level — so it adjusts automatically depending on how sure the model is."

Q: What's the difference between frequency penalty and presence penalty?

"Frequency penalty cares about *how many times* something's been repeated — the more repeats, the bigger the discouragement. Presence penalty is simpler — it just checks *has this been said before, yes or no* — and nudges toward new material either way."

Q: When would you use a low vs. high temperature?

"Low temperature for anything that needs to be accurate and consistent — like code or factual Q&A. High temperature for anything creative — like brainstorming or storytelling — where you want variety over precision."

Q: Why would you use stop sequences instead of just max tokens?

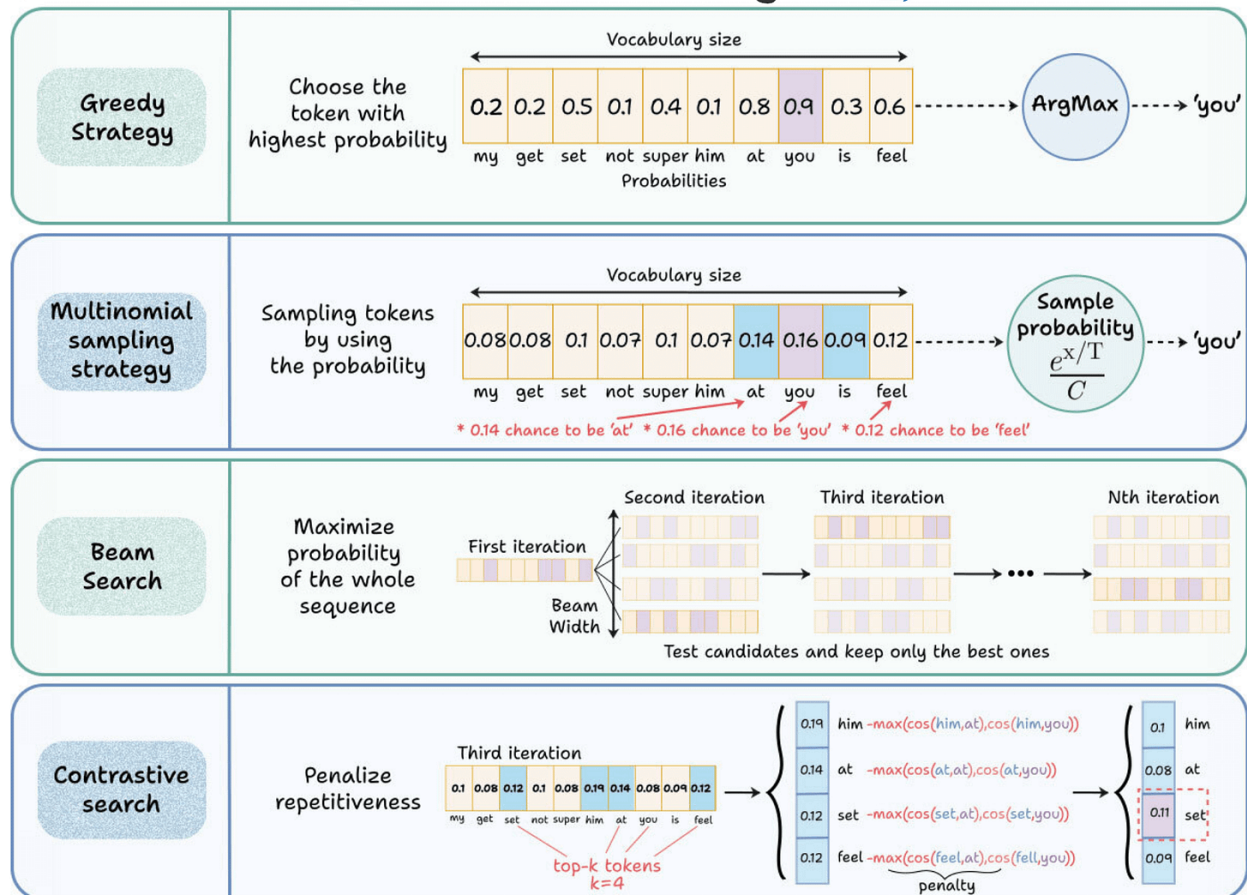
"Max tokens just cuts things off after a certain length, which can chop a sentence in half. Stop sequences let you end generation cleanly at a meaningful point — like right after a closing bracket in JSON — so the output stays clean and structured."

Q: Can you combine these parameters?

"Yes, they're not exclusive — a typical setup might use a moderate temperature together with top-p to balance creativity and coherence, plus stop sequences to keep the output clean."

4 LLM Text Generation Strategies

4 LLM Text Generation Strategies mcp.DailyDoseofDS.com



"When an LLM generates text, it predicts one word at a time, not the whole sentence at once. But having probabilities for the next word isn't enough — we need a strategy to actually decide which word to pick. I like to think of this like **planning a road trip, one town at a time**, where at each town, you have several roads to choose from, each with a different 'promise' score."

1. Greedy Search

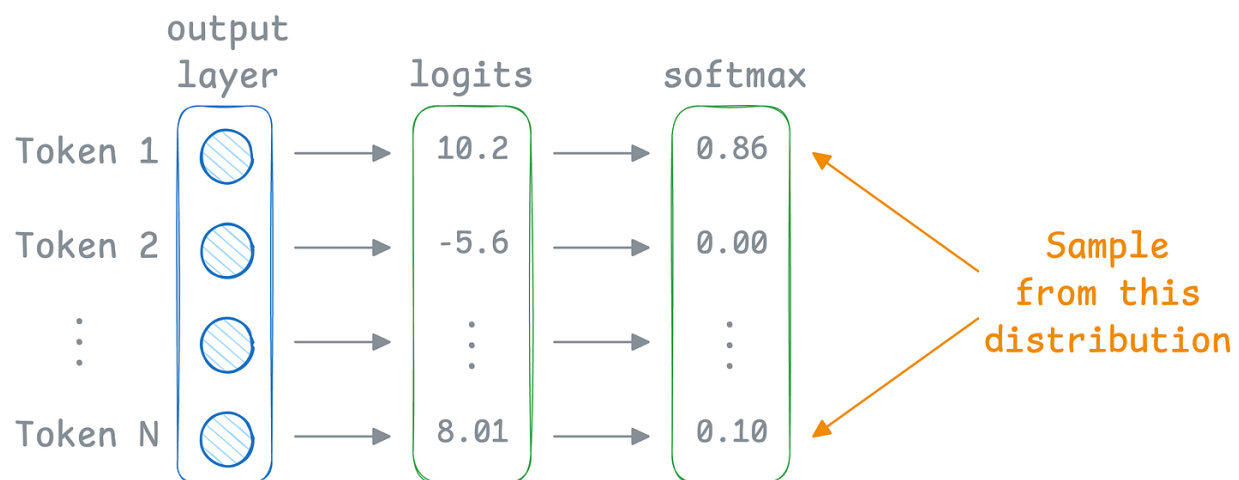
The naive approach greedily chooses the word with the highest probability from the probability vector, and autoregresses. This is often not ideal since it leads to repetitive sentences.

Explanation: Always picks the single **highest-probability** word at each step. Fast and simple, but leads to repetitive, dull output.

Analogy: "It's like always taking the road that *looks* best right this second — the shiny, wide, freshly-paved one — without thinking ahead. You keep taking the 'obvious' road at every town, and you end up going in loops or hitting dead ends, because you never explored alternatives."

2. Multinomial Sampling

Instead of always picking the top token, we can sample from the probability distribution available in the probability vector.



Explanation: Instead of always taking the top word, **we randomly sample based on the probability distribution** — better words have a higher chance but it's not guaranteed. Controlled by **temperature**: higher = more random/creative, lower = more predictable.

Analogy: "This is like spinning a weighted wheel at each town — good roads get bigger slices of the wheel, so you'll usually pick a decent road, but sometimes you'll take a surprising detour. Turn up the 'temperature' and the wheel becomes more evenly spread out (more adventurous trips); turn it down and you almost always pick the safe road."

3. Beam Search

Both approach 1 and approach 2 have a problem. They only focus on the most immediate token to be generated. **Ideally, we care about maximizing the probability of the whole sequence, not just the next token.**

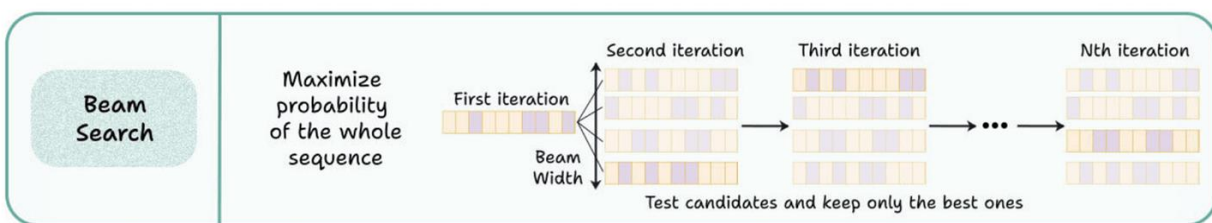
To maximize this product, you'd need to know future conditionals (what comes after each candidate).

But when decoding, we only know probabilities for the next step, not the downstream continuation.

Beam search tries to approximate the true global maximization:

- At each step, it expands the top k partial sequences (the beam).
- Some beams may have started with less probable tokens initially, but lead to much higher-probability completions.
- By keeping alternatives alive, beam search explores more of the probability tree.

This is widely used in tasks like machine translation, where correctness matters more than creativity.



Explanation: Greedy and sampling only look one step ahead. Beam search keeps multiple candidate paths alive at once (say, top 3-5), expands all of them, and at the end picks whichever full path has the best overall score. Great for tasks needing accuracy, like translation.

Analogy: "Instead of committing to one road immediately, you send out **3-4 scout cars** down different roads at the same time. Some scouts might start on a road that doesn't look amazing but eventually leads to the best destination. At the end of the trip, you compare all the scouts' routes and pick whichever one had the best overall journey — not just the best first turn."

4. Contrastive Search

This is a newer method that balances fluency with diversity.

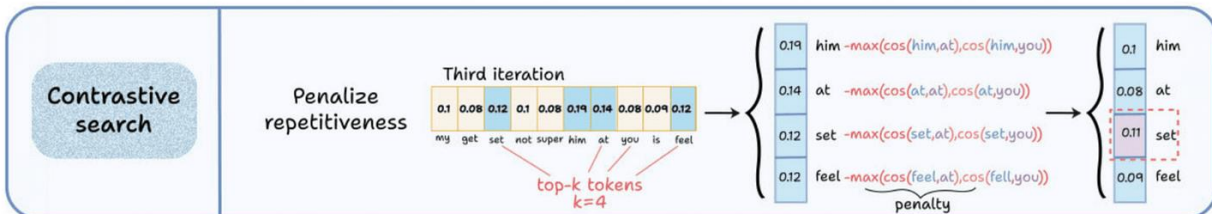
Essentially, it penalizes repetitive continuations by checking how similar a candidate token is to what's already been generated to have more diversity in the output.

- At each step, the model considers candidate tokens.
- Applies a penalty if the token is too similar to what's already been generated.

- Selects the token that balances probability and diversity.

This way, it also prevents "stuck in a loop" problems while keeping coherence high.

It's especially effective for longer generations like stories, where repetition can easily creep in.



Explanation: A newer method balancing fluency and diversity. It penalizes words too similar to what's already been generated, preventing repetition — especially useful in long text like stories.

Analogy: "This is like driving with a rule: 'don't take a road that looks too much like one I already drove.' You still prefer good roads, but you actively avoid backtracking or circling the same neighborhood — keeping the trip fresh and moving forward instead of getting stuck looping through the same town."

Closing summary (with analogy)

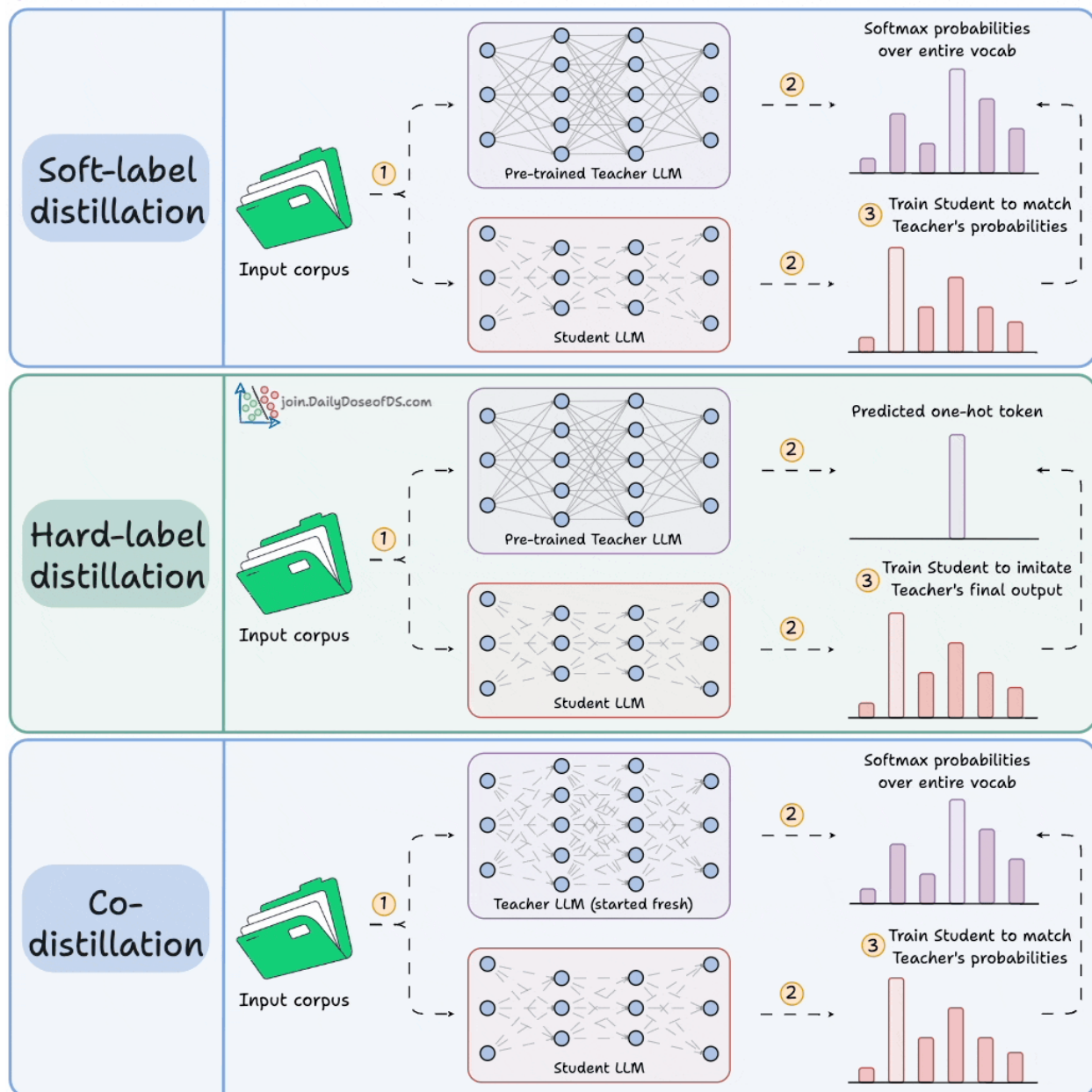
"So overall — greedy takes the closest exit even if it's a dead end, sampling adds a bit of adventurous randomness, beam search sends out scouts to find the best overall route, and contrastive search avoids retracing your steps. The right strategy depends on the destination: beam search when you need the *correct* route (like translation), and sampling or contrastive search when you want a *creative*, non-repetitive journey (like storytelling)."

3 Techniques to Train An LLM Using Another LLM

LLMs don't just learn from raw text; they also learn from each other:

- Llama 4 Scout and Maverick were trained using Llama 4 Behemoth.
 - Gemma 2 and 3 were trained using Google's proprietary Gemini.
- Distillation helps us do so, and the visual below depicts three popular techniques.

3 Techniques to Train an LLM using Another LLM



The idea is to transfer "knowledge" from one LLM to another, which has been quite common in traditional deep learning ([like we discussed here](#)).

Distillation in LLMs can happen at two stages:

1) Pre-training

- Train the bigger Teacher LLM and the smaller student LLM together.

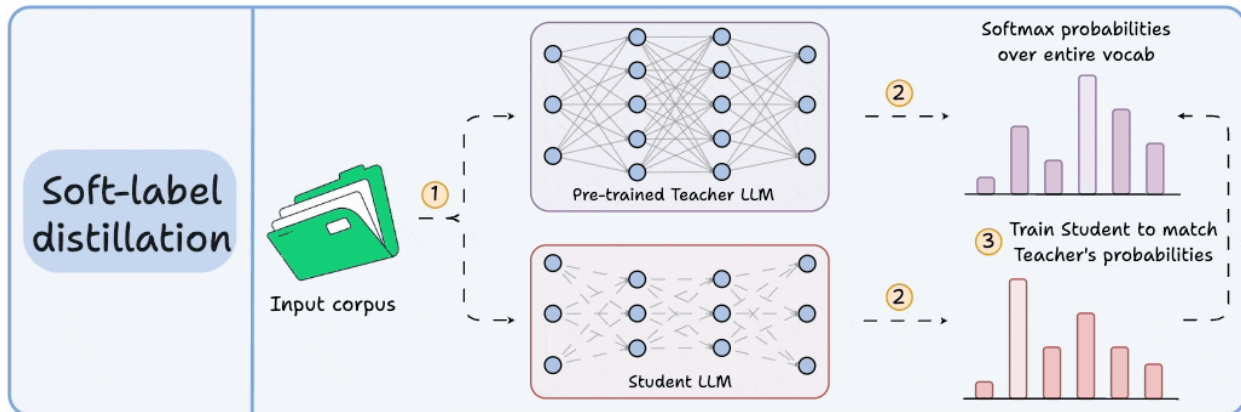
- Llama 4 did this.

2) Post-training:

- Train the bigger Teacher LLM first and distill its knowledge to the smaller student LLM.

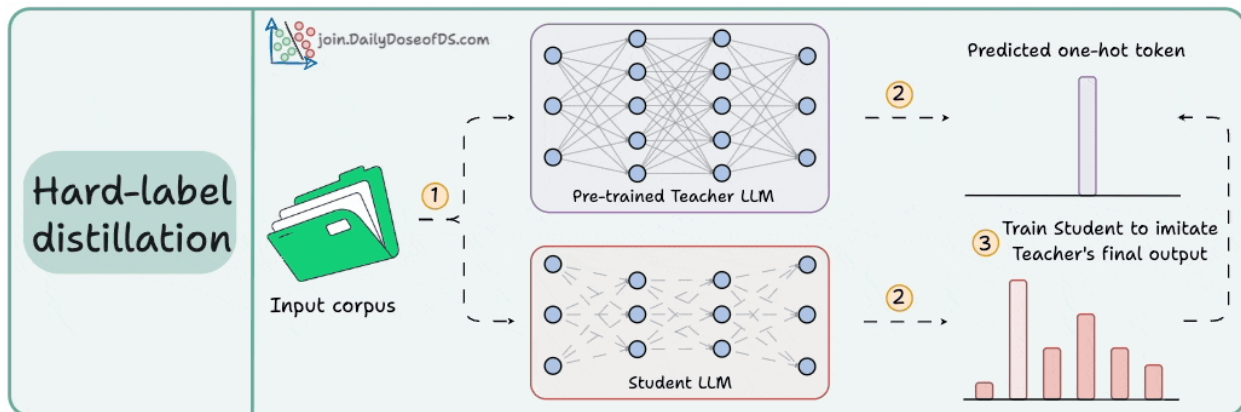
- DeepSeek did this by distilling DeepSeek-R1 into Qwen and Llama 3.1 models. You can also apply distillation during both stages, which Gemma 3 did. Here are the three commonly used distillation techniques:

1) Soft-label distillation:



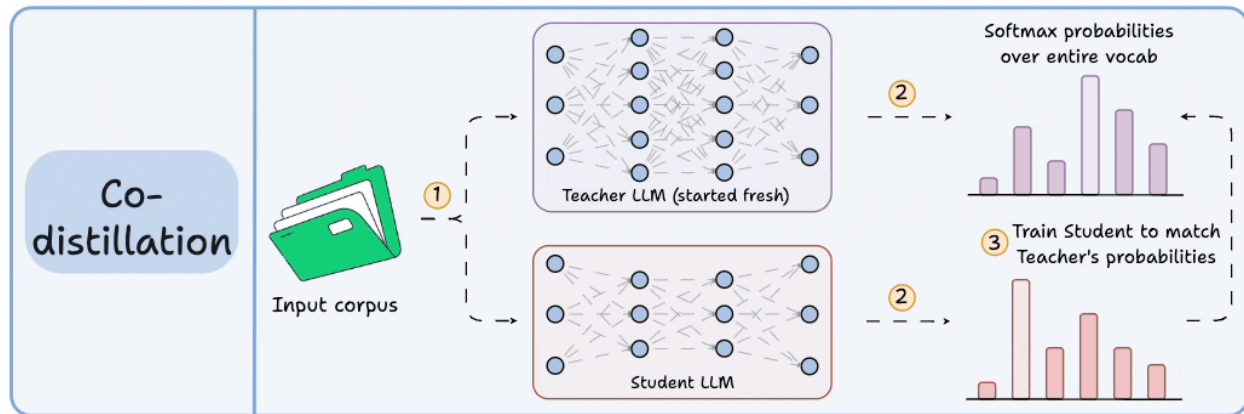
- Use a fixed pre-trained Teacher LLM to generate softmax probabilities over the entire corpus.
- Pass this data through the untrained Student LLM as well to get its softmax probabilities.
- Train the Student LLM to match the Teacher's probabilities. Visibility over the Teacher's probabilities ensures maximum knowledge (or reasoning) transfer. However, you must have access to the Teacher's weights to get the output probability distribution. Even if you have access, there's another problem! Say your vocab size is 100k tokens and your data corpus is 5 trillion tokens. Since we generate softmax probabilities of each input token over the entire vocabulary, you would need 500 million GBs of memory to store soft labels under float8 precision. The second technique solves this.

2) Hard-label distillation



- Use a fixed pre-trained Teacher LLM to just get the final one-hot output token.
- Use the untrained Student LLM to get the softmax probabilities from the same data.
- Train the Student LLM to match the Teacher's probabilities. DeepSeek did this by distilling DeepSeek-R1 into Qwen and Llama 3.1 models.

3) Co-distillation



- Start with an untrained Teacher LLM and an untrained Student LLM.
 - Generate softmax probabilities over the current batch from both models.
 - Train the Teacher LLM as usual on the hard labels.
 - Train the Student LLM to match its softmax probabilities to those of the Teacher.
- Llama 4 did this to train Llama 4 Scout and Maverick from Llama 4 Behemoth.
Of course, during the initial stages, soft labels of the Teacher LLM won't be accurate.
That is why Student LLM is trained using both soft labels + ground-truth hard labels.

"Distillation is basically transferring knowledge from a bigger, smarter model (Teacher) to a smaller model (Student), so the Student can perform almost as well without needing the same size or compute. It's like a student learning from an expert teacher instead of learning everything from scratch by reading raw textbooks."

"So basically, the idea here is — you don't always need to train an LLM from scratch using raw text. You can also make one LLM learn from another LLM that's already trained. This is called **distillation** — think of it like a smaller model 'copying' knowledge from a bigger, smarter model.

A few real examples — Llama 4's smaller models, Scout and Maverick, were trained using a much bigger model called Behemoth. Google's Gemma models were trained using Gemini. And DeepSeek did something similar too.

Now, there are basically **3 ways** people do this distillation."

1. Soft-label distillation

"In this one, the teacher model doesn't just say 'this is the answer' — it shows its **full confidence spread**. Like, 'I'm 70% sure it's this word, 20% sure it's that word, 10% something else.' And the student model tries to copy that entire confidence pattern, not just the final answer.

It's great because the student learns a lot more nuance — almost like learning *how* the teacher thinks, not just *what* it says.

But the problem is — storing that much information for every single word, across a huge dataset, is insanely expensive. Like, we're talking hundreds of millions of GBs kind of expensive. So it doesn't scale well."

2. Hard-label distillation

"This one's simpler — the teacher just gives the **final answer**, no confidence breakdown, nothing. Just 'the answer is this.' The student trains to match that.

It's way lighter on memory, way easier to scale. And this is actually what DeepSeek did — they took their big R1 model and distilled it down into smaller models like Qwen and Llama 3.1."

3. Co-distillation

"This one's a bit different — here, **both** the teacher and student start off untrained. They're basically learning together, side by side. The teacher trains normally on the real correct answers, and the student tries to copy the teacher's outputs as it goes.

But since early on the teacher itself isn't great yet, the student also keeps checking against the actual ground-truth answers too — so it's not blindly trusting the teacher.

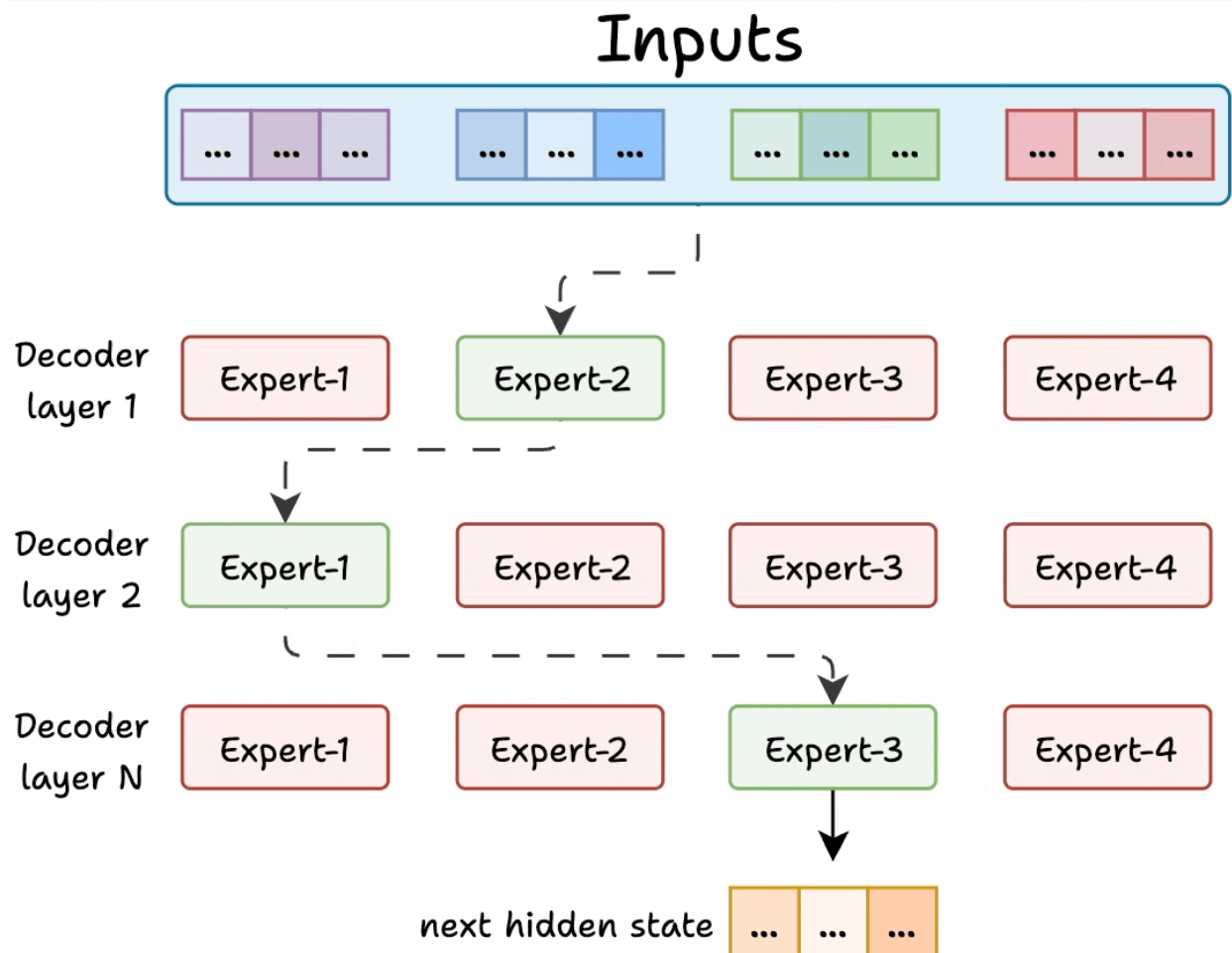
This is what Llama 4 did to train Scout and Maverick from Behemoth."

Quick way to wrap it up

"So basically — **soft-label** gives the richest info but is super heavy on memory, **hard-label** is lightweight and just transfers the final answer, and **co-distillation** trains both models together in **real time**. Which one you use really depends on how much compute and memory you have, and how much depth of knowledge you want to transfer."

Transformer vs. Mixture of Experts (MoE) in LLMs

- Transformer uses a feed-forward network.
- MoE uses experts, which are feed-forward networks but smaller compared to that in Transformer.
During inference, a subset of experts are selected. This makes inference faster in MoE.
Also, since the network has multiple decoder layers:



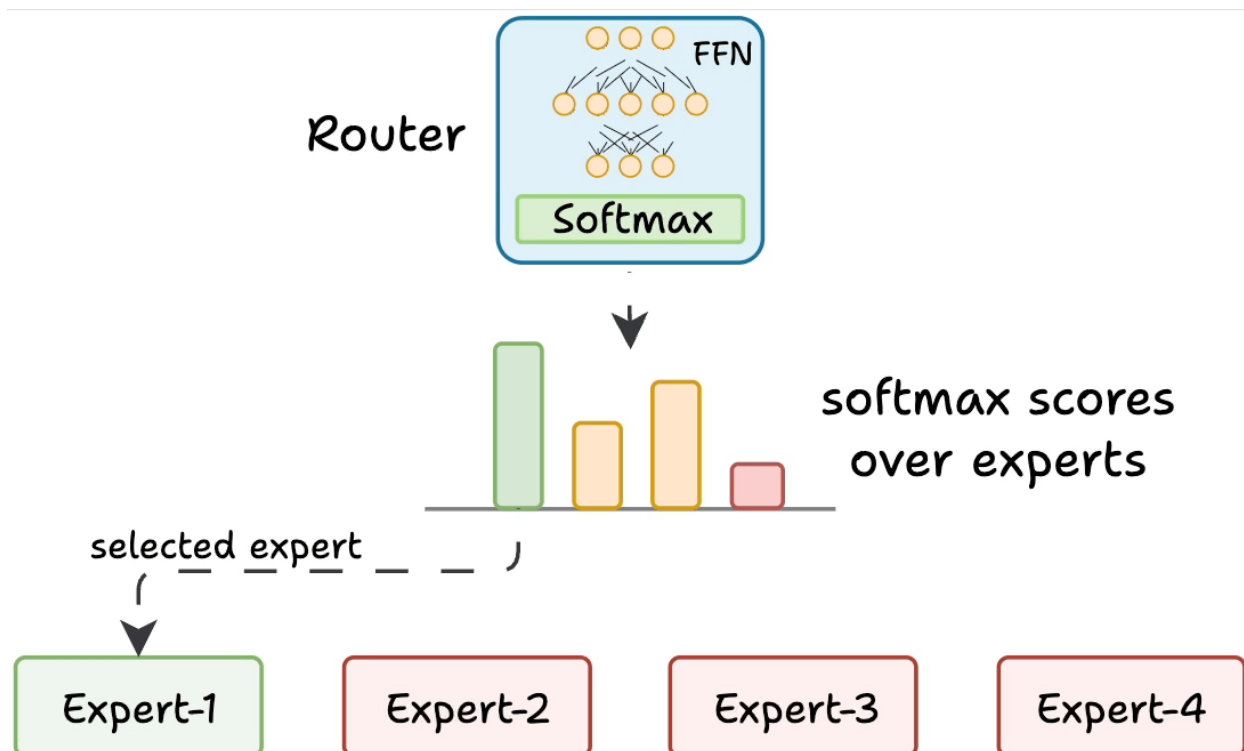
- the text passes through different experts across layers.
- the chosen experts also differ between tokens.

But how does the model decide which experts should be ideal?

The router does that.

The router is like a multi-class classifier that produces softmax scores over experts. Based on the scores, we select the top K experts.

The router is trained with the network and it learns to select the best experts.



Core Idea

The main difference between a regular Transformer and an MoE model is in the **decoder block**:

- **Transformer** → uses one feed-forward network (FFN) per layer.
- **MoE** → replaces that single FFN with multiple smaller FFNs called "**experts**." During inference, only a subset of these experts is actually used.

This selective activation makes MoE models **faster at inference**, even though they have more total parameters.

How Experts Get Selected — The Router

- A component called the **router** decides which experts to use.
- The router acts like a multi-class classifier — it produces softmax scores over all experts.
- Based on these scores, the **top K experts** are selected for each token.
- Since there are multiple decoder layers, tokens can pass through **different experts at each layer**, and different tokens can pick different experts.
- The router is trained along with the rest of the network.

Challenge 1: Expert Imbalance (Rich-Get-Richer Problem)

Early in training, all experts look similar, so one gets picked somewhat randomly (say Expert 2). Because it gets used, it gets a bit better → gets picked again → gets even better → keeps getting picked. Meanwhile, other experts remain under-trained.

Fix (2-part solution):

1. Add **noise** to the router's output so other experts get a fair chance at higher scores.
2. Set all logits **except the top K** to $-\infty$ — after softmax, these become zero, forcing variety in selection over time.

Challenge 2: Uneven Token Load

Some experts may end up processing far more tokens than others, causing imbalance again.

Fix: Set a **limit on how many tokens** an expert can process. Once an expert hits its limit, extra tokens are routed to the **next-best expert** instead.

Key Outcome

- MoE models have **more total parameters** than a standard Transformer, but only a **fraction are active** at any time (since only top K experts are used).
- This leads to **faster inference** despite the larger overall model size.
- **Example real-world model:** Mixtral 8x7B by Mistral AI.

Simple Explanation with Analogy (Interview-Ready)

Analogy: A Hospital With Specialist Doctors

"Think of a **Transformer** like a hospital with just **one general physician** — every patient (token), no matter what problem they have, goes to this same doctor. It works, but the doctor has to know a bit of everything, and it's not maximally efficient.

Now, **MoE** is like a hospital with a team of **specialist doctors** — a cardiologist, a dermatologist, an orthopedic, etc. When a patient comes in, a **receptionist** (this is the router) looks at their symptoms and directs them to the 2-3 most relevant specialists, instead of sending them to every doctor in the building.

So instead of one generalist handling everyone, you've got many specialists, but each patient only sees a couple of them. That's why it's faster — you're not activating the whole hospital for every patient, just the relevant experts."

How I'd Explain It Out Loud in an Interview

"So the key difference between a regular Transformer and a Mixture of Experts model is in the feed-forward part of the decoder. A normal Transformer has one feed-forward network that every token passes through. MoE replaces that with multiple smaller feed-forward networks called 'experts' — and for each token, only a few of these experts actually get used.

There's a **router** — think of it like a receptionist — that looks at each token and decides which experts should handle it. It gives a score to every expert, kind of like a classification problem, and then picks the top K experts based on those scores.

Now, this creates a couple of real challenges during training.

First problem — early on, since all experts start off pretty similar, one expert might randomly get picked a bit more. Because it gets more practice, it becomes slightly better, so it keeps getting picked again and again — kind of like the 'rich getting richer.' Meanwhile, other experts barely get used and stay undertrained.

To fix this, two things are done — you add some **random noise** to the router's scoring so other experts also get a fair shot at being picked, and you also force only the top K experts to be usable at all — everything else gets zeroed out. This keeps forcing variety over time.

Second problem — even if experts are picked more evenly, some might still end up handling way more tokens overall than others. So there's usually a **cap** — like, 'this expert can only handle this many tokens' — and once it hits that limit, extra tokens get redirected to the next best expert instead.

End result — yeah, MoE models technically have way more parameters overall, but since only a few experts activate per token, inference ends up being **faster** compared to a dense Transformer of similar size. Mixtral 8x7B is a well-known model built this way."

Quick Closing Line for Interviews

"So basically — Transformer is like one generalist doctor seeing every patient, while MoE is like a hospital with a team of specialists, where a router acts like a receptionist directing each patient to just the right couple of specialists. This makes MoE more efficient at inference, even though it holds way more total knowledge/parameters overall."

Prompt Engineering

Complete Notes — Theory, Analogies, Code (Euri API) & Interview Q&A

Simple-language notes • Flow Diagrams • Comparison Tables • Not Bookish

Sources: AI Engineering Guidebook (DailyDoseofDS) • `prompt_engineering_notes_euri.md`
(raffeemdai/LangChain_LangGraph) • current prompting best-practice guides

1. What Is Prompt Engineering?

A prompt is simply the text/instructions you send to an LLM. Prompt Engineering is the skill of designing and refining that text so the model gives you the output you actually want — not a rough guess at what you meant.

💡 **Analogy:** A prompt is like a steering wheel for the LLM. You're not changing the engine (the model's trained weights) — you're just steering the same engine in a more precise direction. Small wording changes can completely shift where the output ends up.

A good prompt helps the model:

- Think step-by-step instead of jumping to conclusions
- Follow constraints (length, format, tone)
- Stay focused on exactly what you asked
- Avoid shallow, generic answers

It's the fastest, lowest-effort lever you have to improve LLM output — no training, no GPUs, just better instructions.

1.1 Prompt Design vs Prompt Engineering

These sound similar but aren't quite the same thing:

	Focus	Example
Prompt Design	Clarity & simplicity — phrase the ask clearly	"What is the capital of France?"
Prompt Engineering	Sophisticated control — few-shot, reasoning nudges, persona, format, temperature	"You are an expert nutritionist... respond in a friendly tone, as a table..."

Same underlying task, two different levels of effort:

- Design (simple): "Give me a business idea for a tech startup."
- Engineering (few-shot + guided reasoning): "Here are 2 examples of good startup ideas: [...]. Based on these, give me a new idea focused on remote team collaboration."

Engineering-style prompts are longer to write but give far more specific, controllable output — this trade-off is the whole game of prompt engineering.

1.2 5 Basic Rules for Writing Good Prompts

1. Assign a Persona — "You are an expert personal finance advisor..." This primes the model's tone, vocabulary and depth.
2. State the Core Task & Subtasks — say exactly what you want done, broken into parts if needed.
3. Provide Context — background info the model has no way of knowing on its own.

4. Explain the Expected Output — format (table/list/prose), length, tone.
5. Describe Limitations — what to avoid, what timeframe/data to focus on.

Example combining all 5 rules in one prompt:

```
You are an expert personal finance advisor. Explain different types of investment options available in India. Assume the audience has basic banking knowledge. Give the explanation as a table. Focus only on data from the last 5 years.
```

Notice how each of the 5 rules maps to one sentence: persona → task → context (audience) → output format (table) → limitation (last 5 years only).

2. Setting Up Euri (Used in Every Code Example Below)

All code examples in this document use the free Euri API — an OpenAI-compatible endpoint. Because it's OpenAI-compatible, it works directly with the official openai Python SDK, and also with LangChain's ChatOpenAI class, without any special adapter.

Step 1 — Get a free key

Sign up at euron.one/euri and copy your API key. The free tier gives roughly 200,000 tokens/day with no credit card, across models like Gemini, Llama, and GPT-family mirrors.

Step 2 — Install packages

```
pip install openai langchain langchain-openai python-dotenv
```

Step 3 — Create a .env file (never hardcode API keys in code)

```
EURI_API_KEY=your_api_key_here
EURI_BASE_URL=https://api.euron.one/api/v1/euri
EURI_MODEL=gemini-2.5-flash
```

Step 4 — A tiny reusable helper (euri_client.py)

```
# euri_client.py
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()

client = OpenAI(
    api_key=os.getenv('EURI_API_KEY'),
    base_url=os.getenv('EURI_BASE_URL'), # https://api.euron.one/api/v1/euri
)
MODEL = os.getenv('EURI_MODEL', 'gemini-2.5-flash')

def ask(messages, temperature=0.7):
    """messages = list of {'role': 'system'/'user'/'assistant', 'content': '...'}"""
    resp = client.chat.completions.create(
```

```

    model=MODEL,
    messages=messages,
    temperature=temperature,
)
return resp.choices[0].message.content

```

What this code does:

- `load_dotenv()` pulls your API key out of the `.env` file into environment variables — this is the standard way to avoid ever committing a secret key into your code.
- `client = OpenAI(...)` is created ONCE at import time. Every function that calls `ask()` reuses this same client — you never want to reconnect on every single request.
- `base_url` is what redirects the standard OpenAI SDK to Euri's servers instead of OpenAI's — this one line is the entire trick that makes Euri 'OpenAI-compatible'.
- `ask(messages, temperature)` is a thin wrapper: you pass in a list of role-tagged messages, it calls the chat completion endpoint, and returns just the plain text reply (`resp.choices[0].message.content`) — hiding all the boilerplate response-object structure.
- Every example in this document just does `from euri_client import ask` and calls `ask([...])` — you write this file once and reuse it everywhere.

A LangChain version of this same setup (using ChatOpenAI) is shown in Section 7.

3. Core Prompting Techniques

These five techniques form the everyday toolkit. Master these before reaching for anything more advanced.

3.1 Zero-Shot Prompting

Ask the model to do a task with no examples — it relies purely on what it already learned during training.

 **Analogy:** Like asking a well-read colleague a general-knowledge question cold, with no hints — they either know it or they don't.

Zero-Shot

User Question → LLM (uses only trained knowledge) → Answer

```

from euri_client import ask

messages = [{'role': 'user', 'content': 'What is the capital of Japan?'}]
print(ask(messages))
# -> Tokyo

```

What this code does:

- One user message, no examples, no system prompt — the simplest possible call.
- Works great for facts and well-known tasks; gets shaky for tasks needing a specific format or unusual reasoning.

3.2 Few-Shot Prompting

Give the model a few input → output examples before asking your real question. This teaches the pattern/format you want, instead of describing it in words.

💡 Analogy: *Like showing a new employee 2-3 completed sample forms before asking them to fill out a new one — showing beats explaining.*

```
from euri_client import ask

prompt = '''Here are some translations from English to French:
English: "Good morning." -> French: "Bonjour."
English: "How are you?" -> French: "Comment ca va?"

Translate this sentence to French: "What is your name?"
'''

messages = [{'role': 'user', 'content': prompt}]
print(ask(messages))
# -> "Quel est ton nom?"
```

What this code does:

- The prompt bundles 2 worked examples (English → French) directly above the real question.
- The model pattern-matches the format of the examples (short, no extra commentary) and applies that same format to the new input — this is why few-shot is so good at controlling OUTPUT FORMAT, not just content.
- More examples generally help up to a point, but each one costs tokens — 2-5 examples is a common sweet spot.

3.3 Chain-of-Thought (CoT) Prompting

Ask the model to reason step by step instead of jumping straight to the final answer. Breaking a complex problem into smaller logical steps often improves accuracy — especially on math, logic, and multi-step problems.

💡 Analogy: *Like asking someone to 'show their working' on a math test instead of just writing the final number — you (and the model) catch mistakes along the way instead of after the fact.*

Chain of Thought

Query → Step 1 → Step 2 → ... → Step N → Final Output

```

from euri_client import ask

messages = [{
    'role': 'user',
    'content': 'Solve this step by step: What is 12 multiplied by 9?'
}]

print(ask(messages, temperature=0)) # temperature=0 -> deterministic/precise
# -> Step 1: 12 x 9 means adding 12 nine times...
# -> Final answer: 108

```

What this code does:

- The phrase 'step by step' is the entire trigger — it nudges the model to lay out intermediate reasoning before committing to a final number.
- temperature=0 is used here on purpose: for math/logic you want the MOST likely (deterministic) path, not a creative one.

3.4 System Prompting (Setting Behavior / Persona)

A system message sets the model's role/behavior for the whole conversation. It's sent once, before the actual question, and quietly shapes every reply that follows.

💡 Analogy: *Like giving a new call-center agent their briefing before their shift starts ('you are polite, you always confirm the customer's account number first') — said once, followed for the whole shift.*

```

from euri_client import ask

messages = [
    {'role': 'system', 'content': (
        'You are an expert nutritionist. Give healthy eating advice based '
        'on someone's age and lifestyle. Respond in a friendly, supportive tone.'
    )},
    {'role': 'user', 'content': "I'm 28 and work a desk job. Any tips?"},
]

print(ask(messages))

```

What this code does:

- The system message is a separate role from user — most model providers give it slightly higher priority than an ordinary user instruction, which is why it's the right place for persona/tone/behavior rules.
- The actual question ('I'm 28 and work a desk job...') stays in a user message — keep persona/instructions and the real ask cleanly separated.

3.5 Temperature Control (Creativity vs Precision)

temperature controls how random/varied the model's word choices are.

Range	Behavior	Best for
Low (0 – 0.3)	Deterministic, focused, repeatable	Facts, code, math, extraction
High (0.7 – 1.2)	Creative, varied, less predictable	Stories, brainstorming, taglines

💡 **Analogy:** *Temperature is like a dial between 'safe autopilot' and 'improv mode'. Low = the pilot flies the exact same route every time. High = the pilot is allowed to take scenic detours.*

```
from euri_client import ask

creative_reply = ask(
    [{'role': 'user', 'content': 'Write a tagline for a coffee shop.'}],
    temperature=1.2,
)
precise_reply = ask(
    [{'role': 'user', 'content': 'What is 15 * 12?'}],
    temperature=0.0,
)

print('Creative:', creative_reply)
print('Precise :', precise_reply)
```

What this code does:

- Same ask() function, same model — only the temperature argument changes the personality of the output.
- Running the tagline prompt twice at temperature=1.2 will likely give two different taglines; running the multiplication prompt twice at temperature=0.0 will (almost always) give the exact same answer both times.

4. Advanced Reasoning Techniques (For Hard Problems)

These go one level deeper than plain Chain-of-Thought. They're useful specifically when a task needs strong logical or mathematical reasoning and a single CoT pass isn't reliable enough.

4.1 Self-Consistency (a.k.a. Majority Voting over CoT)

CoT is useful but not always consistent — the same question can get different answers on different runs, especially at higher temperature. Self-Consistency embraces this variation instead of fighting it: ask the model multiple times, generate several independent reasoning paths, and pick the most common final answer.

 **Analogy:** *Like asking 5 people to independently solve the same puzzle, then going with whatever answer most of them agree on — you're trusting the crowd, not any single attempt.*

Self-Consistency

Query → run LLM reasoning 5 times (Path 1, Path 2, Path 3, ...)
→ collect each Output → Majority Vote → Final Answer

It doesn't evaluate HOW the reasoning was done — only whether the final answer is consistent across paths. This makes it simple, but it can still confidently agree on a wrong answer if most paths make the same mistake.

```
from euri_client import ask
from collections import Counter

question = 'If a train travels 60 km in 45 minutes, what is its speed in km/h?'
prompt = f'Solve step by step and give a final numeric answer only at the end:\n{question}'

answers = []
for _ in range(5):
    reply = ask([{'role': 'user', 'content': prompt}], temperature=0.8)
    # crude extraction: take the last line as 'the final answer'
    final_line = reply.strip().splitlines()[-1]
    answers.append(final_line)

most_common_answer, count = Counter(answers).most_common(1)[0]
print('All answers:', answers)
print(f'Majority answer ({count}/5 votes):', most_common_answer)
```

What this code does:

- The SAME prompt is sent 5 separate times inside a loop — temperature=0.8 keeps some randomness so the 5 runs can actually differ from each other.
- Each reply's last line is treated as 'the final answer' — a rough but simple way to pull out just the conclusion from a longer step-by-step explanation.

- `Counter(answers).most_common(1)[0]` counts how often each distinct answer appeared and returns the single most frequent one — that's the 'vote'.
- This costs 5x the API calls of a single CoT prompt — you're trading cost/latency for reliability.

4.2 Tree of Thoughts (ToT)

While Self-Consistency varies the FINAL answer (by repeating the whole thing), Tree of Thoughts varies the STEPS of reasoning at each point, and picks the best path overall. At every reasoning step, the model explores multiple possible directions — these branches form a tree, and a separate evaluation step judges which path looks most promising.

💡 Analogy: *Like a chess player considering several possible next moves, mentally evaluating each one a few moves ahead, and only then committing to the strongest line — instead of just playing the first move that comes to mind.*

Tree of Thoughts

Query → generate Path 1, Path 2, Path 3
 → Evaluate all paths → keep Best Path
 → expand further → Final Answer

It's more compute-intensive (multiple LLM calls: generate branches, evaluate them, expand the best one) but in most cases significantly outperforms basic CoT on genuinely hard, multi-step problems.

```
from euri_client import ask

problem = 'You have 3 boxes weighing 4kg, 7kg, and 9kg. Split them into two groups with the smallest possible weight difference.'

# Step 1: generate multiple candidate first moves (branches)
branch_prompt = f'''Problem: {problem}
Propose 2 different possible groupings (branches) as short bullet points, without solving fully yet.'''
branches = ask([{'role': 'user', 'content': branch_prompt}], temperature=0.9)
print('Candidate branches:\n', branches)

# Step 2: ask the model to evaluate and pick the best branch, then solve it
evaluate_prompt = f'''Problem: {problem}
Here are candidate groupings:
{branches}

Evaluate each candidate's weight difference, pick the best one, and give the final answer with the reasoning.'''
final_answer = ask([{'role': 'user', 'content': evaluate_prompt}], temperature=0.2)

print('\nFinal (best path):\n', final_answer)
```

What this code does:

- This is a simplified, 2-call version of ToT: call #1 generates candidate branches (high temperature = more variety), call #2 evaluates them and picks a winner (low temperature = precise judgment).
- Notice the second prompt literally pastes the first call's output (branches) into itself — this is the 'tree' part: the model reasons about its own earlier candidate paths rather than starting fresh.
- A full ToT implementation would repeat this branch → evaluate cycle at multiple depths — this simplified version stops at one level to keep the example readable.

4.3 Quick Comparison

Technique	What varies	What it optimizes for
Chain of Thought	Nothing — a single reasoning pass	Making one answer more transparent / accurate
Self-Consistency	Runs the SAME prompt multiple times	Reliability — majority vote on the final answer
Tree of Thoughts	Explores DIFFERENT reasoning paths at each step	Quality — searches for the best path, not just the most common one

4.4 Bonus: Attentive Reasoning Queries (ARQ)

CoT, Self-Consistency and ToT all improve reasoning, but they still rely on free-form thinking — the model 'thinks aloud' in its own words. That breaks down in long, rule-heavy tasks: give an agent a 2,000-word system prompt full of policies and tone rules, and it typically starts strong, then drifts, forgets instructions from a few turns ago, and eventually breaks a rule it was explicitly told never to break (e.g. promising a refund it wasn't allowed to offer).

💡 Analogy: Free-form CoT is like telling an employee 'think it through' with no checklist — quality depends on their memory and discipline. ARQ is like handing them a structured form to fill in at each decision point: 'What is the customer's issue? Which policy applies? Am I allowed to offer a refund here?' — the form itself forces them to re-check the rules every time.

ARQ solves this by replacing free-form reasoning with explicit, domain-specific questions, encoded as structured keys inside a JSON schema. Before making a recommendation or a tool call, the LLM is prompted to fill in specific structured fields rather than 'think out loud' in prose.

This does two things: (1) it reinstates critical instructions by re-surfacing them at every decision point, keeping the model aligned mid-conversation, and (2) it makes intermediate reasoning auditable and verifiable, since each step is a checkable structured field instead of loose prose.

Reported success rate across 87 test scenarios (from the source research):

Approach	Success Rate
ARQ (structured queries)	90.2%

Approach	Success Rate
CoT (free-form reasoning)	86.1%
Direct response (no reasoning step)	81.5%

This approach is implemented in Parlant, an open-source framework for building instruction-following agents, where ARQs are used inside three modules: the guideline proposer (which behavioral rules apply right now), the tool caller (which external function to use), and the message generator (the final reply). The core lesson generalizes beyond any specific tool: making reasoning explicit, measurable, and domain-aware stops the model from improvising — structure wins over free-form thinking in high-stakes, multi-turn scenarios.

5. Verbalized Sampling

Post-training alignment methods (like RLHF) are designed to make LLMs helpful and safe. A side effect: they unintentionally cause a big drop in output diversity, called mode collapse — the model starts favoring a narrow set of predictable, "safe" responses over other equally valid ones.

💡 **Analogy:** *Imagine a chef who used to cook 50 different dishes, but after years of customers always praising the same 2 'crowd-pleaser' recipes, starts making only those 2 dishes — even though the other 48 recipes were just as good. The chef's range didn't disappear; it just stopped getting used.*

5.1 Why does mode collapse happen? (typicality bias)

During alignment training, human annotators rate different model responses, and the model is then trained to mimic those preferences. But annotators naturally tend to favor answers that are familiar, easy to read, and predictable — even when an equally correct but more creative answer exists. This is typicality bias. The result: the aligned model's probability distribution gets aggressively sharpened toward one or two dominant, highly predictable responses.

Importantly, this isn't irreversible. The model effectively has two personalities after alignment: the original pretrained model that still holds the full rich distribution of possible answers, and the safety-aligned model that has been nudged to strongly favor just the most predictable response.

5.2 How Verbalized Sampling fixes it

Verbalized Sampling (VS) is a training-free prompting strategy — no retraining needed. The core idea: the prompt itself acts like a mental switch. Asking directly ("Tell me a joke") triggers the aligned personality's single, most-reinforced answer. But asking for a distribution instead of an instance ("Generate 5 responses with their corresponding probabilities. Tell me a joke.") forces the model to draw on its full pretrained knowledge instead of collapsing to one safe answer.

💡 **Analogy:** *It's like asking the chef 'give me your top 5 dish ideas with how confident you are in each' instead of just 'cook me something' — the framing itself invites the wider repertoire back out.*

```
from euri_client import ask

# Direct prompting -> usually the same 'safe' joke every time
direct = ask([{'role': 'user', 'content': 'Tell me a joke.'}])

# Verbalized Sampling -> asks for a distribution, not a single instance
vs_prompt = (
    'Generate 5 different jokes with your estimated probability for '
    'each one (as a rough percentage). Format as a numbered list.'
)
diverse = ask([{'role': 'user', 'content': vs_prompt}], temperature=1.0)

print('Direct:\n', direct)
print('\nVerbalized Sampling (diverse):\n', diverse)
```

What this code does:

- The only real change is the PROMPT itself, not any API parameter — this is what makes VS 'training-free': it's pure prompt engineering.
- Asking for 5 responses + probabilities forces the model to articulate its own uncertainty/distribution in words, which in turn pulls from a wider slice of what it learned during pretraining.
- Research cited in the source material reports 1.6–2.1x higher diversity versus direct prompting, with larger models (GPT-4.1, Gemini-2.5-Pro class) benefitting even more — and VS stacks with other techniques like temperature scaling, since it works at the prompt level, not the sampling level.

6. JSON Prompting for LLMs

When you give an LLM an open-ended instruction, it has to guess what "good output" looks like. Sometimes it adds extra commentary, sometimes it skips details, sometimes formatting changes for no reason. The problem isn't the model — it's the lack of structure in the prompt. For tasks like extraction, reporting, automation, or analysis, you need the output to stay consistent every single time. That's where JSON prompting helps.

 **Analogy:** *Natural language instructions are like verbally telling someone how to fill out a form — everyone interprets 'give me the important details' a little differently. A JSON prompt is like literally handing them the blank form with labeled boxes — there's no ambiguity about what goes where.*

6.1 Why JSON works so well

AI models are trained on huge amounts of structured data from APIs and web applications — JSON is close to their 'native language'. When you speak it, they respond with much more precision.

1) Structure means certainty

JSON forces you to think in fields and values, eliminating gray areas and guesswork about what you actually wanted.

2) You control the output shape

Prompting isn't just about what you ask — it's about what you expect back. JSON prompts ensure a consistent structure every time, whether you're generating content, reports, or insights. No more surprises.

3) Reusable templates → scalability, speed, clean handoffs

A JSON prompt can be turned into a shareable template. Teams can plug results directly into APIs, databases, and apps — no manual reformatting — so the whole pipeline stays reliable and fast.

6.2 Euri code — open-ended vs JSON prompt

```
from euri_client import ask
import json

# Open-ended (vague) -- output format can vary run to run
vague_prompt = 'Summarize this email and give key takeaways: ' + email_text
```

```

print(ask([{'role': 'user', 'content': vague_prompt}]))

# JSON prompting -- consistent, parseable output every time
json_prompt = f'''Extract information from this email and respond with
ONLY valid JSON, no extra text, in exactly this schema:
{{
  "sender_intent": string,
  "key_points": [string],
  "action_required": boolean,
  "deadline": string or null
}}

Email:
{email_text}'''

reply = ask([{'role': 'user', 'content': json_prompt}], temperature=0)
data = json.loads(reply) # safely parseable Python dict
print(data['key_points'])
print('Needs action?', data['action_required'])

```

What this code does:

- The `vague_prompt` version leaves format entirely up to the model — one run might return bullet points, the next might return a paragraph.
- The `json_prompt` version explicitly defines the schema (field names + types) and instructs 'ONLY valid JSON, no extra text' — this is the key line that stops the model from adding conversational filler around the JSON.
- `temperature=0` is used again here — for structured extraction tasks you want maximum consistency, not creativity.
- `json.loads(reply)` converts the model's text reply into an actual Python dict — this only works reliably because the prompt forced a strict, predictable format. Skipping the schema instruction would make this line crash unpredictably.

6.3 It's not just about JSON specifically

Good alternatives exist depending on the model: Claude tends to handle XML tags exceptionally well, and Markdown provides structure without much overhead. The underlying principle isn't "always use JSON" — it's "give the model explicit structure instead of open-ended prose," whichever syntax fits your pipeline.

In short: structured prompting is like writing modular code — it brings clarity of thought, makes adding new requirements effortless, and creates better communication with the AI. It's less a one-off technique and more a habit worth building into how you write every production prompt.

7. Prompt Templates (Reusable Prompts in Code)

Hardcoding prompts as raw strings gets messy fast once you have more than a couple of them. Templates let you define a prompt once with {placeholders} and reuse it with different inputs, cleanly.

7.1 Plain Python — a simple template function

```
from euri_client import ask

def explain_template(topic: str, audience: str) -> str:
    return f'Explain {topic} to a {audience} in 2 sentences.'

prompt = explain_template('vector embeddings', '5-year-old')
print(ask([{'role': 'user', 'content': prompt}]))
```

What this code does:

- No library needed — just a normal Python function that returns an f-string. This is the simplest possible 'template': a function with parameters instead of a hardcoded string.
- Good enough for small scripts; gets clunky once you need multi-role messages, few-shot examples baked in, or input validation — that's where LangChain's template classes below help.

7.2 LangChain PromptTemplate + Euri

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate

load_dotenv()

llm = ChatOpenAI(
    model=os.getenv('EURI_MODEL', 'gemini-2.5-flash'),
    api_key=os.getenv('EURI_API_KEY'),
    base_url=os.getenv('EURI_BASE_URL'),
)

template = PromptTemplate.from_template(
    'Explain {topic} to a {audience} in 2 sentences.'
)

chain = template | llm      # LCEL: pipe prompt straight into the model
result = chain.invoke({'topic': 'vector embeddings', 'audience': '5-year-old'})
print(result.content)
```

What this code does:

- ChatOpenAI is pointed at Euri using the same base_url trick from Section 2 — LangChain doesn't need to know Euri exists, it just thinks it's talking to an OpenAI-compatible server.

- `PromptTemplate.from_template(...)` defines the `{topic}` and `{audience}` placeholders declaratively, instead of manually building an f-string.
- `chain = template | llm` uses LangChain's pipe syntax (LCEL) — the template's filled-in output automatically becomes the input to the LLM. You'll see this pattern (`prompt | llm | ...`) constantly in LangChain code.
- `chain.invoke({...})` fills the placeholders with real values and runs the whole pipeline in one call.

7.3 ChatPromptTemplate — system + user roles together

```
from langchain_core.prompts import ChatPromptTemplate

chat_template = ChatPromptTemplate.from_messages([
    ('system', 'You are a concise programming tutor who always gives a code example.'),
    ('human', 'Explain {concept} in Python.'),
])

chain = chat_template | llm
response = chain.invoke({'concept': 'decorators'})
print(response.content)
```

What this code does:

- Unlike plain `PromptTemplate` (single string), `ChatPromptTemplate` builds a full multi-role conversation — here a fixed system persona plus a templated human question.
- Only `{concept}` needs to be supplied at `invoke()` time; the system message stays constant across every call using this template — exactly like the persona idea from Section 3.4, but reusable.

7.4 Few-Shot Template (built-in pattern)

```
from langchain_core.prompts import FewShotPromptTemplate, PromptTemplate

examples = [
    {'input': 'happy', 'output': 'sad'},
    {'input': 'tall', 'output': 'short'},
    {'input': 'fast', 'output': 'slow'},
]

example_prompt = PromptTemplate.from_template('Input: {input}\nOutput: {output}')

few_shot_prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=example_prompt,
    prefix='Give the antonym of each word.',
    suffix='Input: {word}\nOutput:',
    input_variables=['word'],
)

final_prompt = few_shot_prompt.invoke({'word': 'generous'}).to_string()
```

```
print(llm.invoke(final_prompt).content)    # -> 'stingy' (approximately)
```

What this code does:

- This automates exactly what you did manually in Section 3.2 (Few-Shot) — examples is a plain list of dicts, and LangChain formats and stitches them together using `example_prompt`.
- `prefix` is the instruction shown once at the top, `suffix` is where your real question is inserted at the bottom — examples always sit in between, following the 'show, then ask' pattern.
- This scales much better than manually writing few-shot text: add/remove examples by just editing the examples list, no string surgery required.

7.5 Reusable Prompt Library Pattern (real projects)

In real codebases, keep all prompt templates in one file and import them wherever needed — this keeps prompts version-controlled, testable, and separate from business logic.

```
# prompts.py -- keep all templates in one place
from langchain_core.prompts import ChatPromptTemplate

SUMMARIZER_PROMPT = ChatPromptTemplate.from_messages([
    ('system', 'Summarize input text in exactly 3 bullet points.'),
    ('human', '{text}'),
])

TRANSLATOR_PROMPT = ChatPromptTemplate.from_messages([
    ('system', 'You are a professional translator. Only output the translation.'),
    ('human', 'Translate to {language}:\n\n{text}'),
])

# main.py
from prompts import SUMMARIZER_PROMPT, TRANSLATOR_PROMPT

summary_chain = SUMMARIZER_PROMPT | llm
translation_chain = TRANSLATOR_PROMPT | llm

print(summary_chain.invoke({'text': 'LangChain is a framework for building LLM
apps.'}).content)
print(translation_chain.invoke({'text': 'Good morning!', 'language':
'Spanish'}).content)
```

What this code does:

- `prompts.py` holds nothing but templates — no API calls, no business logic. This separation means a prompt tweak never requires touching the code that calls it.
- `main.py` imports whichever templates it needs and pipes each one into the shared `llm` object — you can have as many chains as you have templates, all sharing one model connection.

8. Messages — Organizing a Conversation

A real conversation is more than one prompt/response pair. LangChain represents every turn as a typed Message object instead of a plain string:

Type	Role	Simple meaning
SystemMessage	system	Instructions given once, before the chat starts
HumanMessage	user	What the human typed
AIMessage	assistant	What the model replied
ToolMessage	tool	Result of a tool call, sent back to the model

Why not just plain strings? Different providers expect conversation history in slightly different formats internally — Message classes are one common, provider-agnostic way to write it once and have it work everywhere.

8.1 Euri code — multi-turn conversation with memory

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage

load_dotenv()

llm = ChatOpenAI(
    model=os.getenv('EURI_MODEL', 'gemini-2.5-flash'),
    api_key=os.getenv('EURI_API_KEY'),
    base_url=os.getenv('EURI_BASE_URL'),
)

conversation = [
    SystemMessage(content='You are a friendly math tutor.'),
    HumanMessage(content='What is 12 * 8?'),
]

first_reply = llm.invoke(conversation)
conversation.append(first_reply)  # remember the AI's reply
conversation.append(HumanMessage(content='Now divide that by 4.'))

second_reply = llm.invoke(conversation)
print(second_reply.content)  # correctly uses context from turn 1 -> 24
```

What this code does:

- conversation is just a growing Python list — 'memory' here literally means appending every message (both human and AI) back into the same list before the next call.

- `conversation.append(first_reply)` is the critical line — without it, the second `llm.invoke()` call would have no idea what 'that' refers to in 'Now divide that by 4.'
- Each `llm.invoke(conversation)` call re-sends the ENTIRE conversation history so far — LLMs are stateless between calls; the illusion of memory comes entirely from resending history, not from the model 'remembering' anything on its own.

9. Chaining (LCEL) — The Assembly-Line Mental Model

```
chain = prompt | llm | parser
prompt | llm | parser
```

💡 Analogy: Think of it like an actual assembly line: the prompt template fills in your question → passes it to the LLM → the model's raw answer comes out → an optional parser cleans that raw text into something usable (like a Python dict or a plain string).

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt_template = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful geography assistant.'),
    ('user', 'What is the capital of {country}?')
])

chain = prompt_template | llm | StrOutputParser()
result = chain.invoke({'country': 'France'})
print(result)
```

What this code does:

- The `|` (pipe) operator chains components together — output of the left side becomes input to the right side, just like piping commands in a Unix terminal.
- `StrOutputParser()` at the end simply extracts the plain string content from the model's response object, so result is a clean string instead of an `AIMessage` object you'd otherwise have to unwrap with `.content`.
- This 3-piece chain (`prompt | llm | parser`) is the most common LangChain pattern you'll see in real projects — everything from Section 7 (templates) and Section 8 (messages) plugs into this same pipe syntax.

10. Image Prompting (Bonus)

Prompt engineering isn't only for text — image-generation models respond well to a similar structured approach. A typical structure:

```
[main subject, action, mood],  
[art form, art style, artist references],  
[additional settings: lighting, colors, framing]
```

Example:

```
Lion roaring in a dense forest, angry mood, other animals in the distant  
background, realistic, colorful, 8K
```

Things worth specifying explicitly: art medium (painting, ink, 3D render), camera/lens, resolution (4K/8K), lighting, and material (metal, cloth, glass, wood) — the same "give explicit structure instead of vague prose" principle from Section 6 (JSON Prompting) applies here too.

11. Prompt Engineering vs Fine-tuning vs RAG

If you're building a real-world LLM app, you almost never use a raw model straight out of the box. You typically need one (or a combination) of these three levers:

- Prompt Engineering — steers the model at inference time, using only instructions.
- Fine-tuning — adjusts the model's internal parameters (weights) using additional training.
- RAG (Retrieval-Augmented Generation) — retrieves relevant external information at query time and feeds it into the prompt as context.

Prompt engineering and fine-tuning share one fundamental limitation: the model can only use knowledge it already contains from training. Neither can teach the model new facts, private company data, or anything that happened after its training cutoff without RAG's help.

11.1 What Is RAG, in Simple Terms?

RAG breaks down into exactly what its name says:

- Retrieval — fetching relevant information from a knowledge source (a database, document store, or vector index).
- Augmented — enriching the text-generation process with that additional retrieved information.
- Generation — the LLM then writes its answer using both its own knowledge AND the retrieved context.

💡 **Analogy:** If Prompt Engineering is teaching someone HOW to answer better, and Fine-tuning is retraining their whole personality/skillset, RAG is like handing them the right reference book to look something up in, right before they answer — they don't need to have memorized it.

11.2 The Decision Matrix

Two parameters drive the choice: how much external knowledge your task needs, and how much adaptation (behavior, vocabulary, writing style change) it needs.

	Low Model Adaptation	High Model Adaptation
High External Knowledge	RAG	Hybrid (RAG + Fine-tuning)
Low External Knowledge	Prompt Engineering	Fine-tuning

Simple takeaway rules:

- Use RAG when you need a custom knowledge base, but the LLM's existing vocabulary/writing style is already fine.
- Use Fine-tuning when you need to change the model's BEHAVIOR (structure, tone, internal jargon) more than its knowledge.

- Prompt Engineering alone is enough when you don't need a custom knowledge base and don't need to change behavior.
- Use a Hybrid (RAG + Fine-tuning) when your application needs BOTH a custom knowledge base AND a changed model behavior.

Example scenario: an LLM might struggle to summarize company meeting transcripts because speakers use internal jargon/vocabulary the model has never seen — that's a signal you may need fine-tuning (to learn the vocabulary/style) potentially combined with RAG (to pull in the actual meeting content).

11.3 Side-by-Side Comparison

	Prompt Engineering	RAG	Fine-tuning
What changes	Nothing in the model — only the input text	Nothing in the model — external context is added at query time	The model's internal weights, via extra training
Adds new knowledge?	No — only what's already in the prompt/model	Yes — from your retrieval source, updatable anytime	Only what was in the fine-tuning dataset, baked in permanently
Changes behavior/style?	Weakly, via instructions	Not directly — style stays close to the base model	Yes — this is its main strength
Cost & effort	Very low — just better text	Medium — needs a retrieval/vector pipeline	High — needs training data, compute, and re-training for updates
Best for	Quick wins, no custom data needed	Fast-changing or large private knowledge bases	Consistent tone/format/domain jargon across all outputs

12. Quick-Reference Cheat Sheet

```
# Setup (once)
from openai import OpenAI
client = OpenAI(api_key=EURI_API_KEY, base_url=EURI_BASE_URL)

# Zero-shot
client.chat.completions.create(model=MODEL, messages=[
    {'role': 'user', 'content': 'What is the capital of Japan?' }
])

# Few-shot      -> add examples inside the user message before the real question
# Chain-of-thought -> add 'Let's think step by step' / 'Solve step by step'
# System prompt  -> add {'role': 'system', 'content': 'You are a ...'} first
# Temperature    -> 0 = precise/factual, 1+ = creative
# Self-consistency -> call the same prompt N times, majority-vote the answers
# Tree of Thoughts -> generate branches, evaluate them, pick the best path
```

JSON prompting -> define the exact schema, say 'ONLY valid JSON, no extra text'

Concept	One-line memory hook
Prompt	The instructions you send to the model
Zero-shot	No examples — just ask
Few-shot	A few examples first, then the real ask
Chain-of-thought	"Think step by step" before the final answer
Self-consistency	Ask N times, trust the majority answer
Tree of Thoughts	Explore multiple reasoning branches, pick the best
ARQ	Force reasoning into structured fields instead of free text
Verbalized Sampling	Ask for a distribution of answers, not just one
System prompt	Sets persona/behavior for the whole chat
Temperature	0 = precise, 1+ = creative
JSON Prompting	Define the schema; stop the model from guessing format
Prompt Template	Reusable prompt with {placeholders}
Message	Role-tagged turn (system/human/ai/tool) in a conversation
Chain (LCEL)	prompt llm parser — assembly line for LLM calls
RAG	Retrieve external knowledge, then generate using it
Fine-tuning	Retrain the model's weights to change its behavior

13. Interview Questions & Answers

Framed the way an interviewer actually asks — practical and scenario-based, not textbook definitions.

Q1. In your own words, what is prompt engineering — and why does it matter if the model already 'knows' the answer?

It's the skill of designing input text so the model reliably gives you the output you actually want, without touching the model's weights. It matters because knowing something and expressing it usefully are different — an LLM might 'know' the answer but still ramble, use the wrong format, or bury the useful part in fluff. A good prompt is the steering wheel that gets the same engine to the destination faster and more precisely.

Q2. What's the practical difference between prompt design and prompt engineering, and when would you not bother with the more sophisticated version?

Prompt design is just phrasing your ask clearly — 'What is the capital of France?' Prompt engineering layers on techniques like few-shot examples, persona, reasoning nudges, and output-format constraints for more control. I wouldn't reach for full prompt engineering on a one-off, low-stakes question — the extra time isn't worth it. I would reach for it in production systems where consistency and format matter every single time.

Q3. Walk me through when you'd use zero-shot vs few-shot prompting.

Zero-shot when the task is common enough that the model has almost certainly seen the pattern during training — general facts, well-known formats. Few-shot when you need a SPECIFIC output shape or style that's ambiguous from instructions alone — showing 2-3 examples removes that ambiguity far more reliably than describing the format in words. Rule of thumb: if you find yourself over-explaining the format in prose, switch to showing examples instead.

Q4. Why does adding 'let's think step by step' actually improve accuracy on math/logic tasks?

It forces the model to generate intermediate reasoning tokens before the final answer, instead of jumping straight to a (often wrong) guess. Since each token the model generates is conditioned on everything generated before it, having correct intermediate steps in the context genuinely increases the odds the final answer follows logically from them — it's not just cosmetic.

Q5. If Chain-of-Thought already makes the model reason step by step, why do we need Self-Consistency on top of it?

Because a single CoT pass can still land on the wrong answer, especially at higher temperature — you get one reasoning path and you're stuck with whatever it concludes. Self-Consistency runs the same prompt several times, collects several independent reasoning paths, and takes the majority answer — trading extra API calls for higher reliability, similar to asking multiple people the same question and trusting the consensus.

Q6. How is Tree of Thoughts actually different from just running Self-Consistency with more samples?

Self-Consistency reruns the WHOLE prompt independently each time and only compares final answers at the end — the individual reasoning paths never interact. Tree of Thoughts branches WITHIN a single problem-solving process: it generates multiple next-steps at each stage, evaluates which is most promising, and only continues down the strongest branch — closer to a search algorithm than to

repeated independent sampling. It's more expensive but tends to outperform CoT/Self-Consistency on genuinely complex, multi-step problems.

Q7. Your team built an agent with a long system prompt full of rules, and it keeps breaking rules mid-conversation. What's happening, and what's one way to fix it?

This is the exact problem ARQ (Attentive Reasoning Queries) targets — with long, rule-heavy system prompts, models tend to start strong, then drift, and eventually violate an instruction they were given at the very start. Free-form CoT doesn't fully fix this because reasoning is still open-ended prose the model might skip or gloss over. The fix is to replace free-form reasoning with structured, explicit queries — force the model to fill in specific fields like 'which policy applies here?' before it's allowed to respond, re-surfacing the rules at every decision point instead of hoping they're remembered.

Q8. What is 'mode collapse' and why would a perfectly well-aligned, safe model actually be a downside for something like creative brainstorming?

Mode collapse is when alignment training (like RLHF) over-sharpens the model's output distribution toward the 1-2 most 'typical', predictable responses — a side effect of human annotators naturally preferring familiar-sounding answers during training (typicality bias). For factual Q&A this is often fine or even desirable. For brainstorming or creative tasks, it's a real downside: you keep getting the same 'safe' joke or the same generic tagline, even though the model's pretraining actually contains a much richer, more varied set of possible answers that alignment suppressed rather than deleted.

Q9. How does Verbalized Sampling recover that lost diversity without retraining the model?

By changing what you ask for, not how the model is trained. Directly asking for one answer ('Tell me a joke') triggers the aligned model's single most-reinforced response. Asking it to verbalize a DISTRIBUTION instead ('Generate 5 responses with their probabilities') forces the model to represent a spread of possibilities in its own output, which pulls from the broader, more diverse knowledge it still holds from pretraining — even though the underlying weights never changed.

Q10. A teammate says 'just ask the LLM in plain English and it'll figure out the format.' When would you push back on that and insist on JSON prompting instead?

Any time the output feeds into something downstream that needs to parse it reliably — an API, a database, another program. Plain-English prompts leave the OUTPUT format up to the model's guess, which can silently drift between runs (extra commentary one time, missing a field the next). A JSON prompt that defines an exact schema and says 'ONLY valid JSON, no extra text' removes that guesswork, so `json.loads()` on the reply doesn't randomly break in production.

Q11. Is JSON always the right structured format to use?

No — it's the most common default, but not universal. Some models (Claude, for instance) handle XML tags particularly well, and Markdown can give structure with less overhead for simpler cases. The actual principle isn't 'always use JSON' — it's 'give the model explicit structure instead of vague, open-ended prose'; pick the syntax that best fits your model and pipeline.

Q12. Why bother with LangChain's PromptTemplate instead of just using Python f-strings everywhere?

For a single small script, f-strings are fine. Templates start paying off once you have multiple prompts across a real project: they give you named, reusable placeholders, integrate directly into LangChain's pipe syntax (prompt | llm | parser), and keep prompt text separate from business logic — so a prompt

tweak doesn't require touching the code that calls it. It's the same reasoning as using a templating engine instead of string-concatenating HTML.

Q13. In a multi-turn chatbot, why do you have to keep resending the entire conversation history on every single call instead of the model just 'remembering'?

Because LLMs are stateless between API calls — there's no persistent memory inside the model itself between requests. Any illusion of 'memory' in a chat app comes entirely from the application resending the full conversation (system + human + AI messages) as context on every call. That's exactly why, in the LangChain example, you explicitly append the AI's previous reply back into the conversation list before asking the follow-up question — skip that step and the model has no idea what 'that' refers to.

Q14. Give me a real scenario where you'd need RAG instead of just a longer, more detailed prompt.

Whenever the needed information didn't exist at training time or is private — a company's internal documents, this week's news, a customer's account history. You physically cannot put unlimited private data into a prompt (context windows are limited and it doesn't scale), and hardcoding it into the model would require fine-tuning and constant retraining to stay current. RAG retrieves just the relevant slice of that external knowledge at query time and injects it into the prompt — so the model always answers using up-to-date, specific information without ever being retrained.

Q15. An interviewer asks: 'Our chatbot needs to answer questions using our private product docs AND sound exactly like our brand voice, every time.' Which approach(es) would you reach for and why?

Both — this is the textbook Hybrid case on the decision matrix: high external-knowledge need (private product docs → RAG) combined with high model-adaptation need (consistent brand voice → fine-tuning). RAG alone would answer accurately but might not consistently sound 'on-brand'; fine-tuning alone would sound right but wouldn't know product-doc specifics or updates. Prompt engineering (a strong system prompt describing the brand voice) can help on top of either, but for something this demanding, RAG + fine-tuning together is the standard answer.

Q16. Why is fine-tuning described as changing 'behavior' more than 'knowledge' — isn't the model learning new information during fine-tuning?

It does learn from whatever's in the fine-tuning dataset, but that's baked in permanently at training time and doesn't update itself afterward — unlike RAG, which can pull in fresh knowledge on every single query. Fine-tuning's real strength is shifting HOW the model behaves: its tone, vocabulary, output structure, domain-specific style — things that are hard to fully control through prompting alone, and that you want to be consistent across every single response without having to restate instructions every time.

Q17. What's a realistic downside of over-relying on prompt engineering alone for a production system, instead of considering RAG or fine-tuning?

Prompt engineering can only steer knowledge the model already has — it can't teach new facts, and it hits diminishing returns for consistent behavior change (you can ask nicely for a specific tone every time, but it's fragile and can drift). If your task needs current/private information, you need RAG; if it needs deep, consistent behavioral change, you need fine-tuning. Treating prompt engineering as the only lever means you'll eventually hit a wall that better instructions simply can't solve.

Q18. If you were mentoring a junior engineer who just learned prompt engineering, what's one habit you'd tell them to build early?

Treat prompts like code, not throwaway text — version them, test them against real examples, and separate prompt text from application logic (the `prompts.py` pattern). The techniques (few-shot, CoT, JSON schemas) matter, but the habit that actually scales is discipline: knowing exactly what output format you expect, writing it down explicitly, and checking it consistently — rather than tweaking wording by feel every time something looks slightly off.